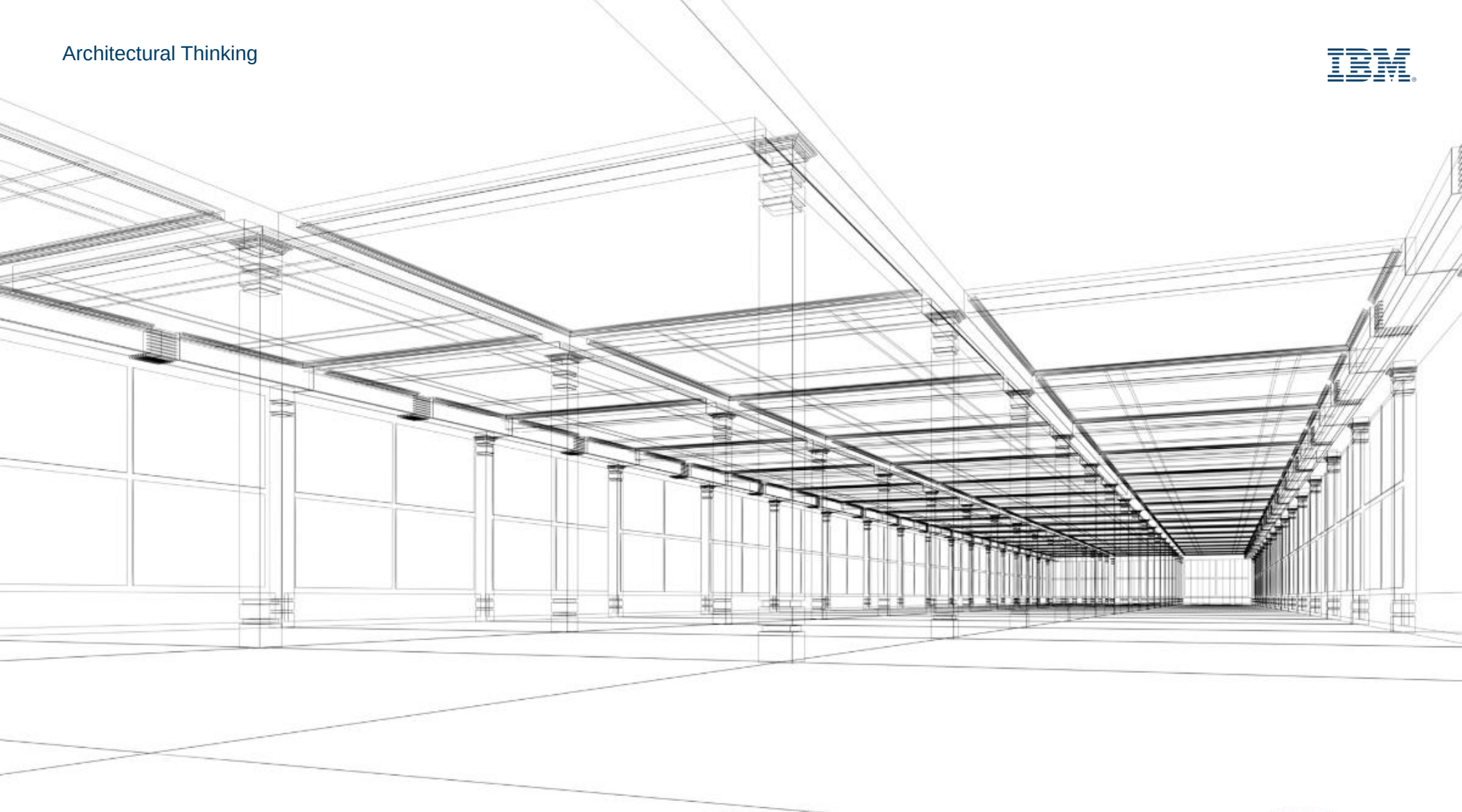




Agile for Architects





Part 1

➤ Introduction and objectives

- Agile - foundational concepts
- Flavours of Agile
- Architecting Solutions in Agile Environment
- Envisaging Initial Architecture

Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- Summary and References



At the end of this module, you should be able to:

- Form an opinion on value and applicability of agile within complex IT landscapes
- Understand how to apply agile methodologies
- Understand role of an architect within agile projects
- Be able to effectively contribute to the agile projects
- Understand what the “initial architecture” looks like
- Recognise characteristics of modern architectural styles
- Understand how DevOps fits into bigger picture



Part 1

- Introduction and objectives
- **Agile - foundational concepts**
- Flavours of Agile
- Architecting Solutions in Agile Environment
- Envisaging Initial Architecture

Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- Summary and References

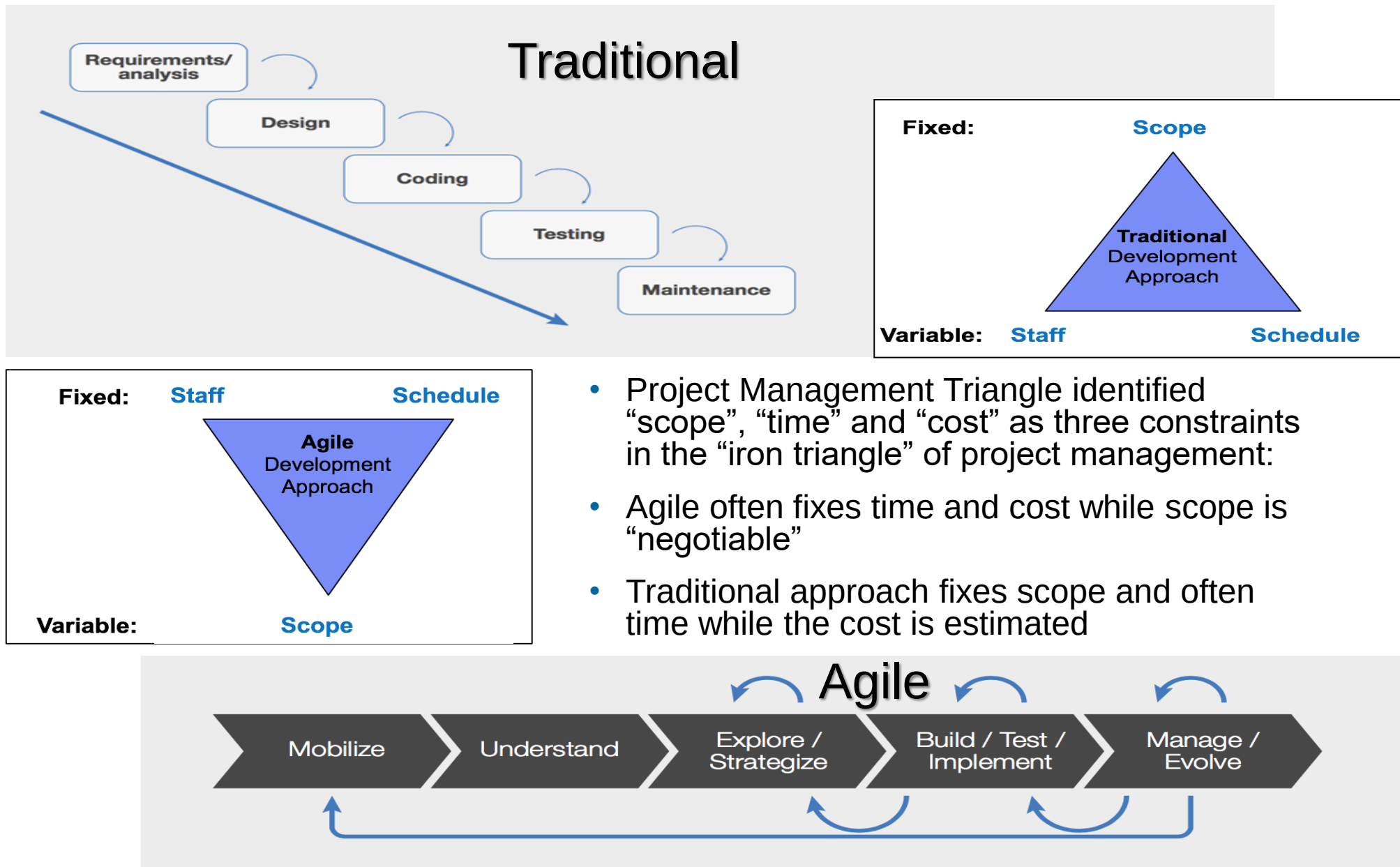


The Agile movement is not anti-methodology, in fact many of us want to restore credibility to the word methodology. We want to restore a balance. We embrace modelling, but not in order to file some diagram in a dusty corporate repository. We embrace documentation, but not hundreds of pages of never-maintained and rarely-used tomes. We plan, but recognize the limits of planning in a turbulent environment. Those who would brand proponents of XP or SCRUM or any of the other Agile Methodologies as "hackers" are ignorant of both the methodologies and the original definition of the term hacker.

— Jim Highsmith, History: The Agile Manifesto

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.



When do we use Agile? – Consider three client scenarios



Scenario 1	Scenario 2	Scenario 3
An airline would like to introduce kiosk based check-in channel at the major airports. Integration services with their core booking system, supporting their existing online check-in system, is to be re-used.	A large telecom wants to replace their legacy CRM and Billing platform. Better customer care and flexibility in rolling out new products to the market are seen as critical to their future growth. Best of breed COTS packages are selected, but not much more beyond that has been done.	A software company developed a HR software for one of their clients. The company retained IP for the software and is now planning to “productise” it as there is a significant market opportunity to sell their HR package as a COTS product.
Agile methods		Traditional methods
Low to medium complexity		High complexity
Requirements are exploratory		Requirements are known
Concept of “minimum viable product” is highly flexible		“Minimum viable product” must have 80% or more of the full scope
Client culture that tolerates “unknown”		Client culture that demands predictability and order
Client has budget to “explore possibilities” and develop POCs		Client has business case with firm scope assumptions



Agile Principle

1	Customer satisfaction by early and continuous delivery of valuable software
2	Welcome changing requirements , even in late development harnessing customer competitive advantage
3	Working software is delivered frequently (from a couple of weeks to a couple of months with preference to shorter timescale)
4	Close, daily cooperation between business people and developers
5	Projects are built around motivated individuals , who should be trusted
6	F2F conversation is the best form of communication (co-location)
7	Working software is the principal measure of progress
8	Sustainable development, able to maintain a constant pace
9	Continuous attention to technical excellence and good design
10	Simplicity —the art of maximizing the amount of work not done—is essential
11	Best architectures, requirements, and designs emerge from self-organizing teams
12	Regularly, the team reflects on how to become more effective, and adjusts accordingly



Recognising that change is hard and needs to be worked at

People

- Identify the **right** people
- **Commit** the right people to projects and prioritise their work
- **Empower** the Product Owners to make **decisions**
- Shift the mindset to focus on agile **values**
- Embrace **transparency**: support people who are confronted by it

Scope

- Set a **clear vision**
- **Prioritise** the portfolio based on **value**
- **Stop projects** when the ROI starts to diminish
- **Small is good** (MVPs & frequent releases)
- **Scope is flexible**
 - One in / One out;
 - Or increase cost and time



Part 1

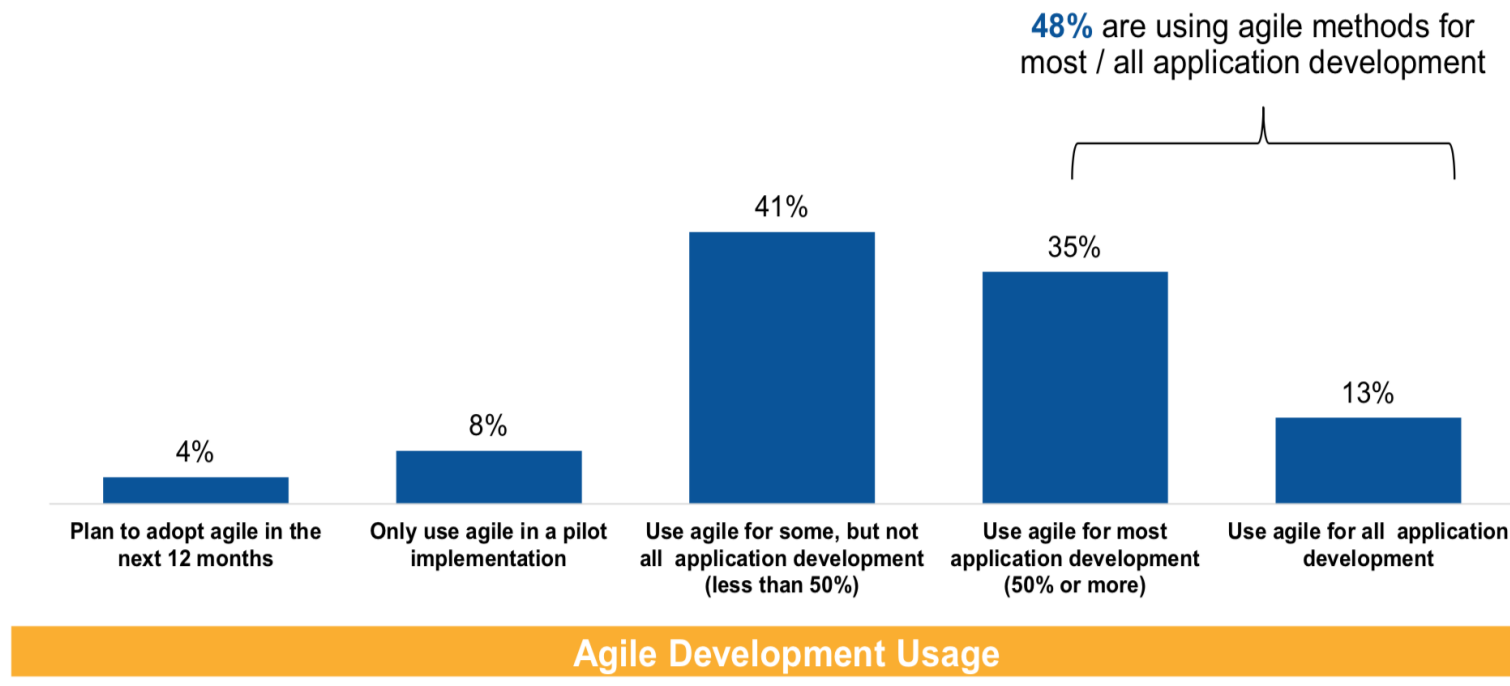
- Introduction and objectives
- Agile - foundational concepts
- **Flavours of Agile**
 - Architecting Solutions in Agile Environment
 - Envisaging Initial Architecture

Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- Summary and References



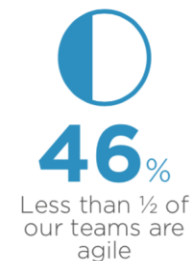
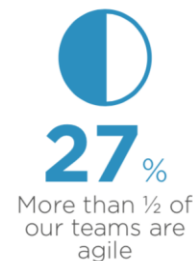
Agile methods: Less than half of global organizations have adopted Agile to a large extent



From Gartner Research Circle report "Agile in the Enterprise 2018"

Percentage of Teams Using Agile

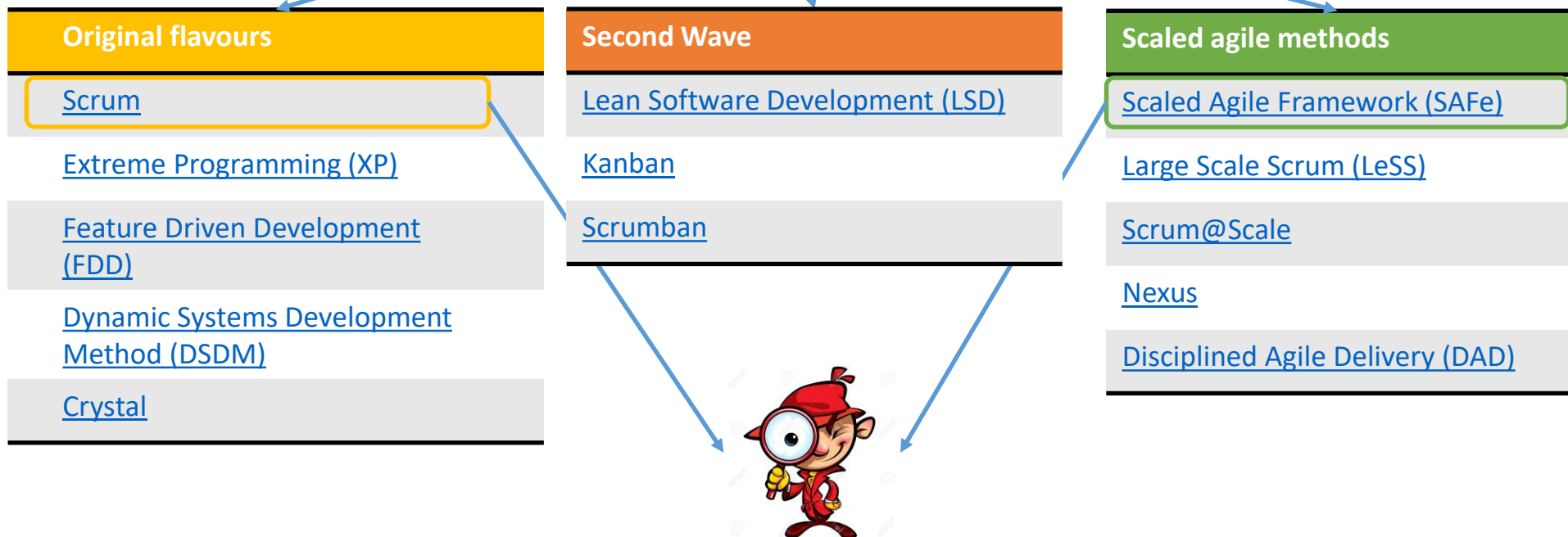
52% of respondents stated that more than half of teams in their organizations are using agile practices.



From VersionOne 12th Annual State of Agile Report



Flavours of Agile



Scrum and SAFe are widely used so lets inspect them further

Scrum – Key Building Blocks

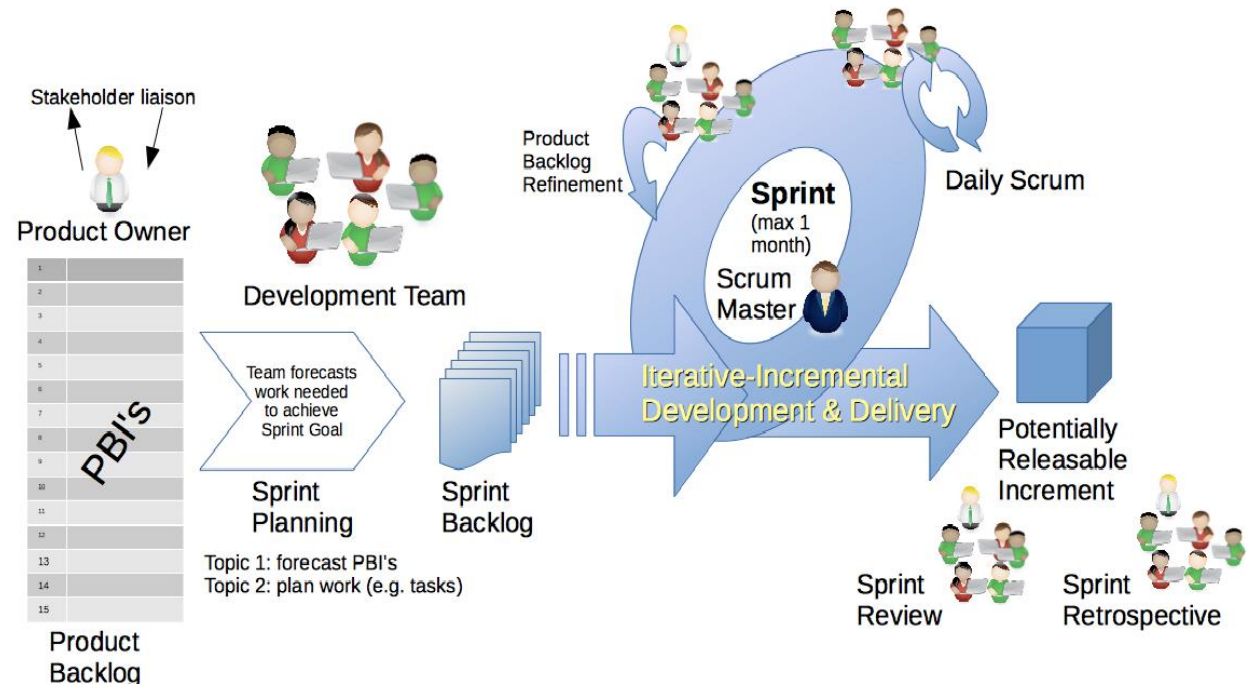


Key Roles

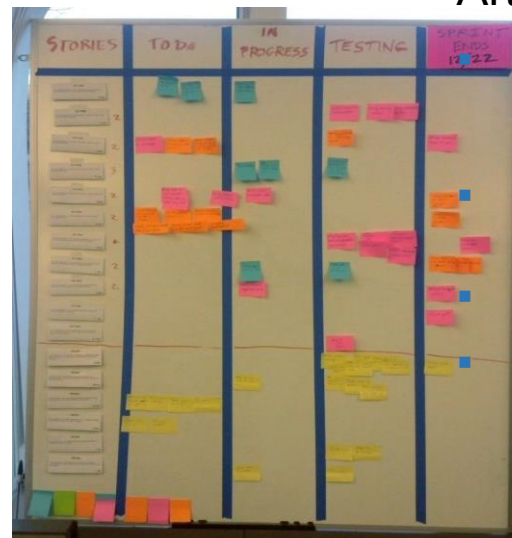
- **Product owner** – voice of customers
- **Delivery team** – multidisciplinary, self-organising, three to 9 members
- **Scrum master** – facilitator and roadblock remover

Workflow

- **Sprint** – Time-boxed unit of development – one week to one month
- **Sprint planning** – scrum team discuss and agree on the scope of work
- **Daily scrum (or stand-up)** – What was done yesterday / plan for today / any roadblocks
- **Sprint review** – playback to stakeholders
- **Sprint retrospective** – reflection / lessons learnt / improvement plan



Artefacts



Product backlog – ordered list of requirements in form of user stories, use cases or any other form the team decides to be useful

Sprint Backlog - list of work the development team must address during the next sprint

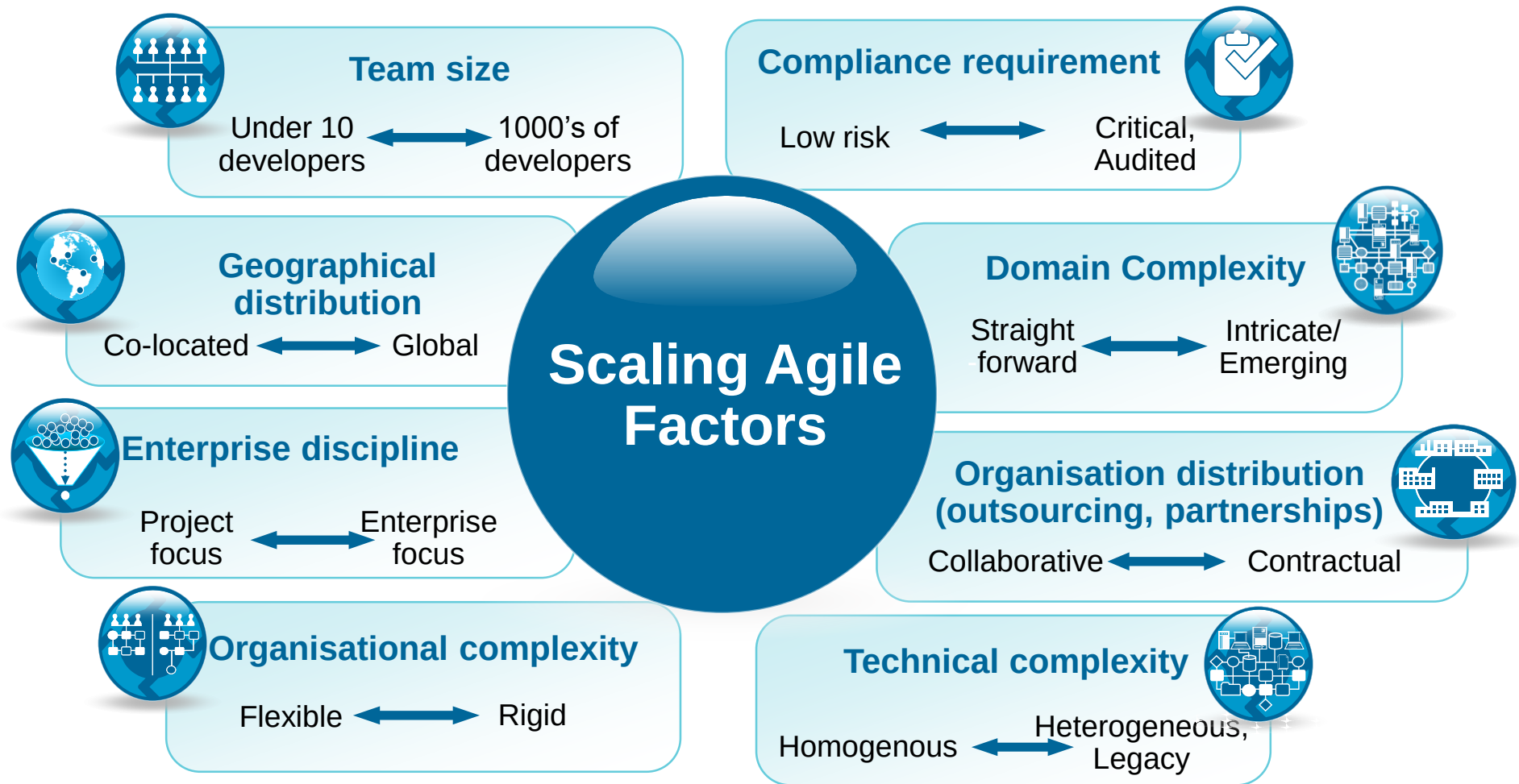
Scrum Board – means of tracking sprint status

Product Increment – potentially releasable increment is the sum of all the product backlog items completed during a sprint, integrated with the work of all previous sprints.

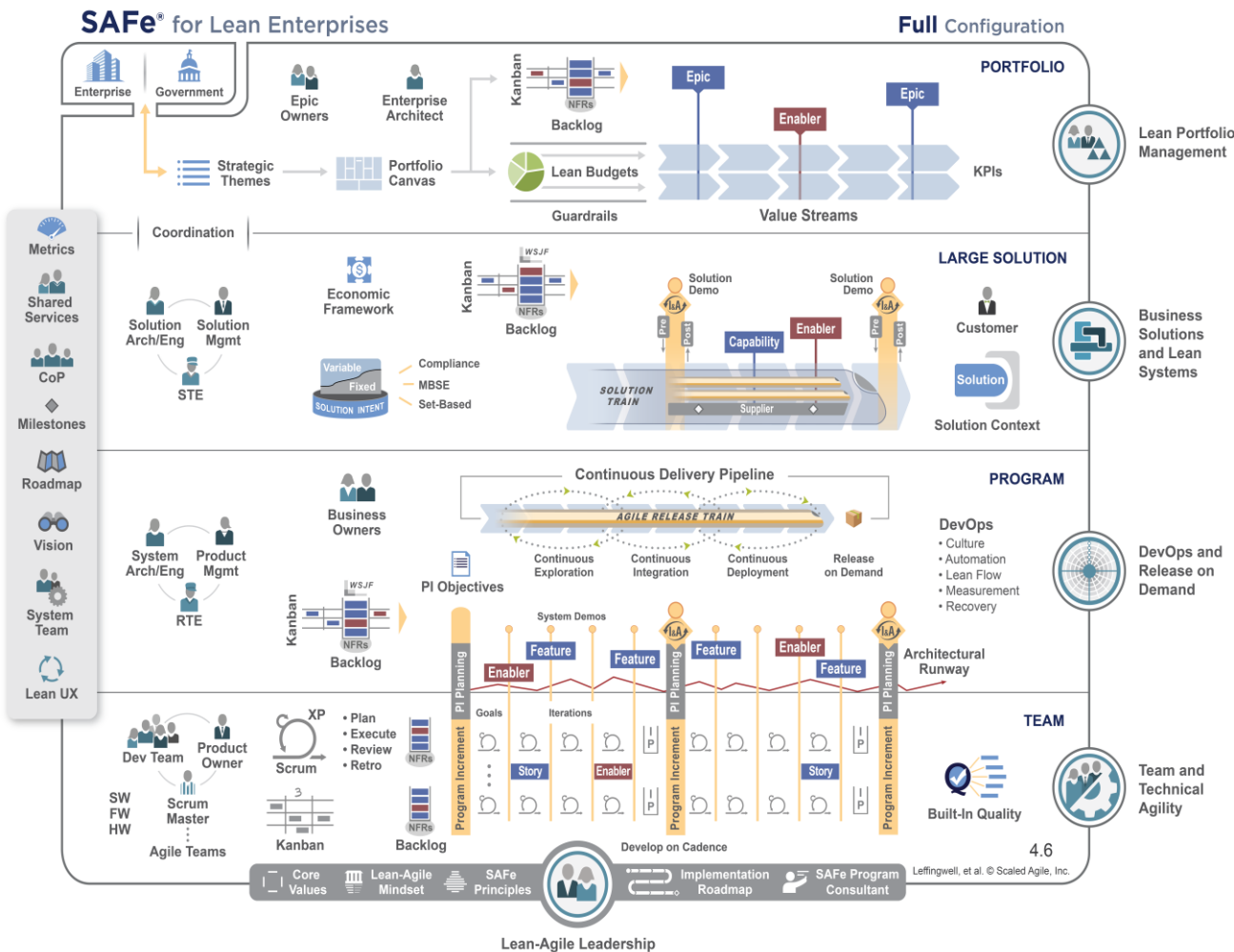


- **Scalability** to large products/projects that require coordination of many scrum teams
 - Drives introduction of scaled agile variations.
- **Geographically dispersed or part-time** team members
 - Somewhat mitigated by modern collaboration tools.
- Team members with very **specialized skills**
 - Scrum, developers should have T-shaped skills, allowing them to work on tasks outside of their specialization.
- Products with many external **dependencies**
 - Given short sprints and detailed planning at the sprint level external dependencies, such as coordination with other teams, can lead to delays.
- Products that **are mature or legacy or with regulated quality control**
 - In Scrum, product increments should be fully developed and tested in a single sprint; products that need large amounts of regression testing or safety testing (e.g., medical devices or vehicle control) for each release are less suited to short sprints than to longer waterfall releases.

How to scale up Agile from the first pilot project towards an Agile enterprise?



SAFe® 4.6 for Lean Enterprises



Portfolio level:

- Identify and organize around Value Streams
- Communicate enterprise strategy with Strategic Themes
- Budgeting and governance on cross-cutting initiatives

Large Solution Level:

- Implement Agile solution trains around a common mission for large solutions
- Align and coordinate ARTs and suppliers to collaborate like a single team

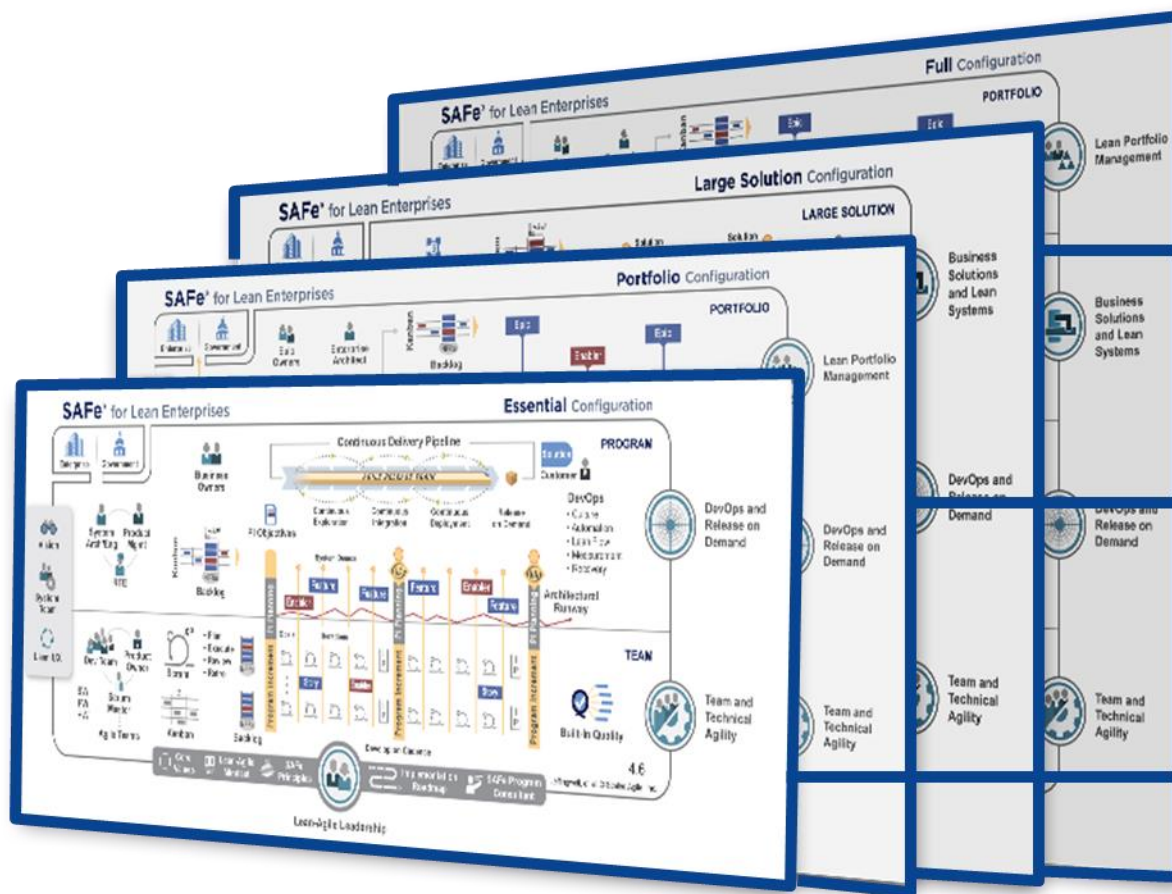
Program level:

- Implement Agile release trains around a common mission
- Apply cadence (heartbeat) and synchronization
- Product management, system architecture

Team level:

- Delivery of user stories - value creation and quality
- End-to-end responsible for software
- Improvements and innovation

SAFe can be configured to meet the needs of a specific enterprise



Full Configuration

Large Solution Configuration

- Brings the Business Solutions and Lean Systems Engineering competency supporting multiple Agile Release Trains (ARTs) and suppliers.

Portfolio Configuration

- Applies Lean Portfolio Management competency to organise development through one or more value streams.

Essential Configuration

Introduces core competencies:

- Lean-Agile Leadership,
- Team and Technical Agility,
- DevOps and
- Release on Demand to the enterprise



Part 1

- Introduction and objectives
- Agile - foundational concepts
- Flavours of Agile
- **Architecting Solutions in Agile Environment**
- Envisaging Initial Architecture

Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- Summary and References

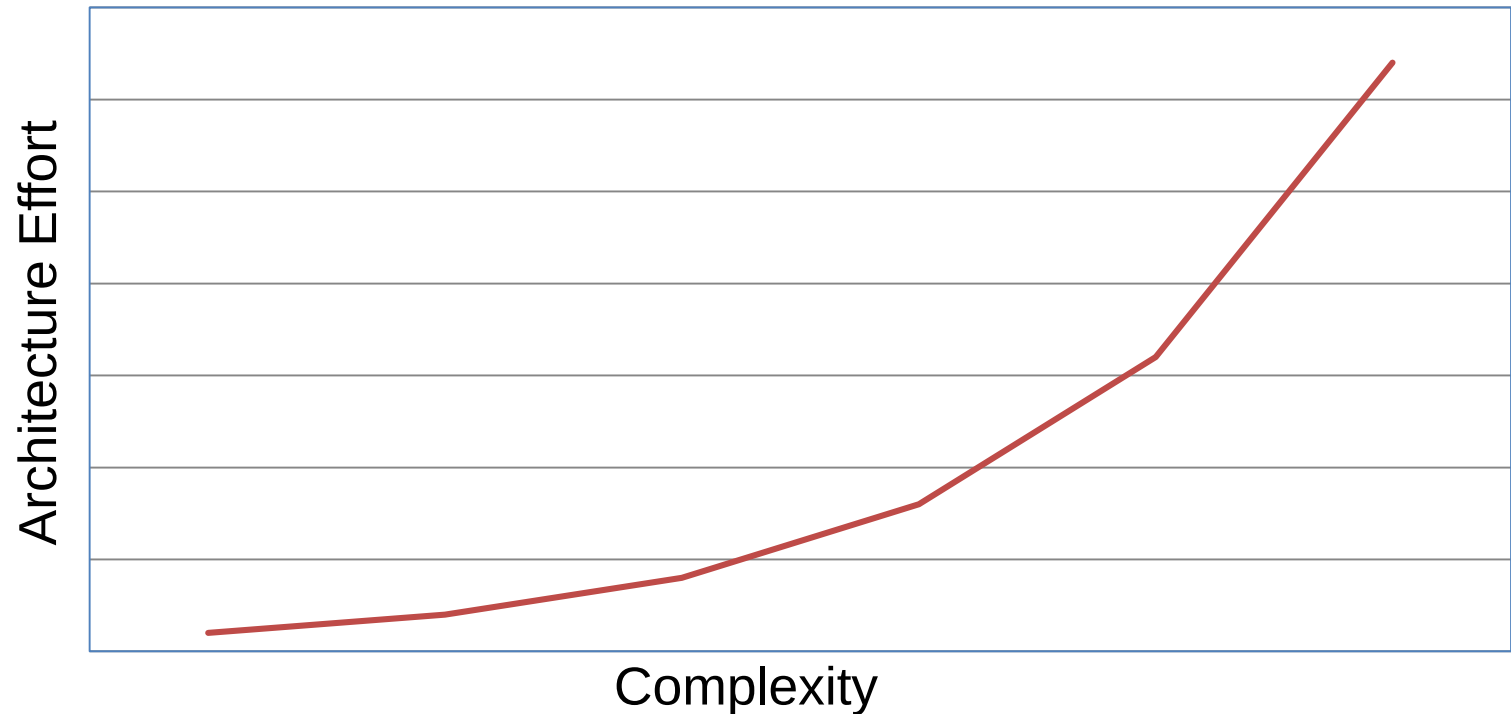
How much architecture is enough?



Myths

Agile projects don't need architecture to be successful

Agile projects need less architecture than "conventional" projects

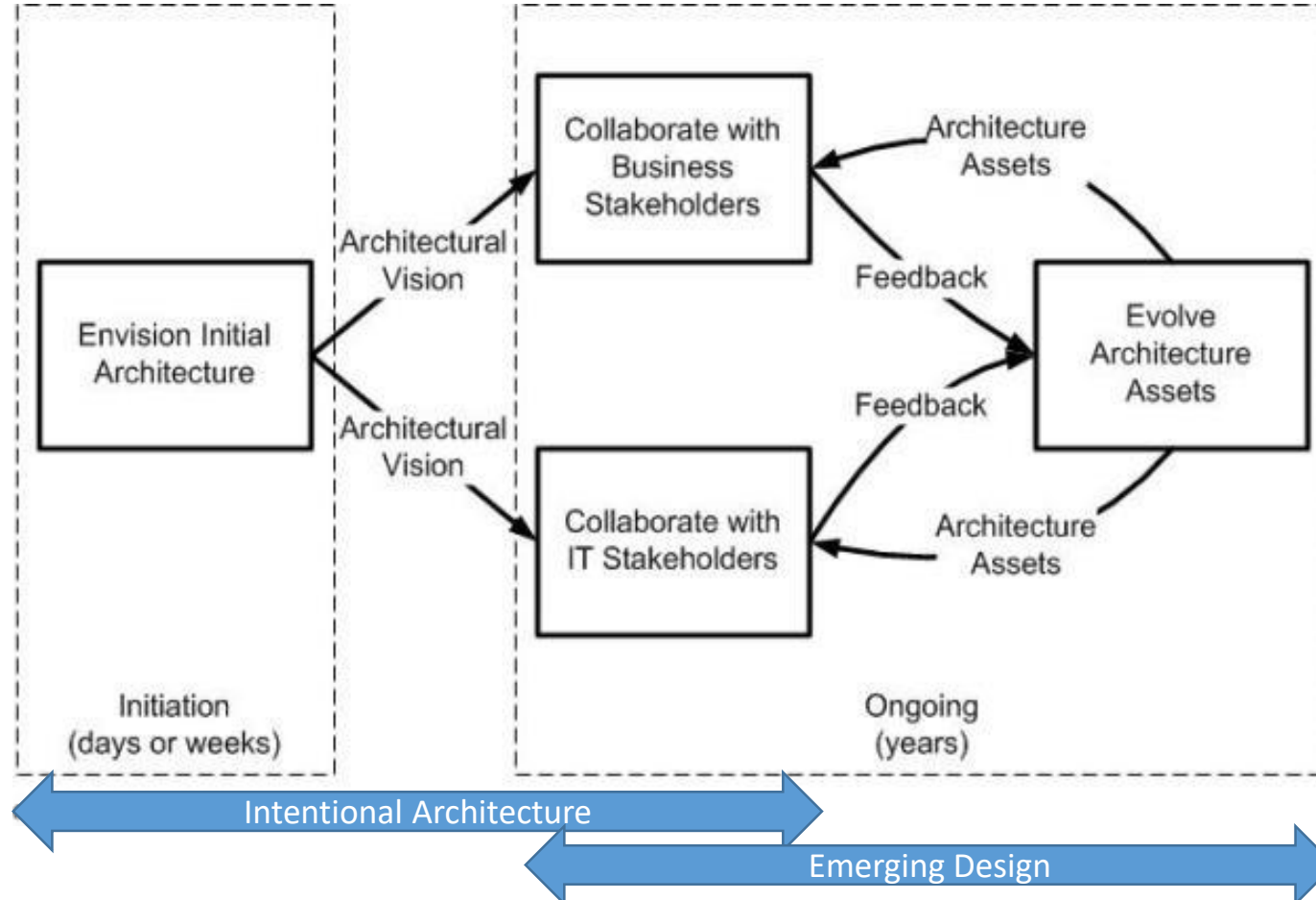


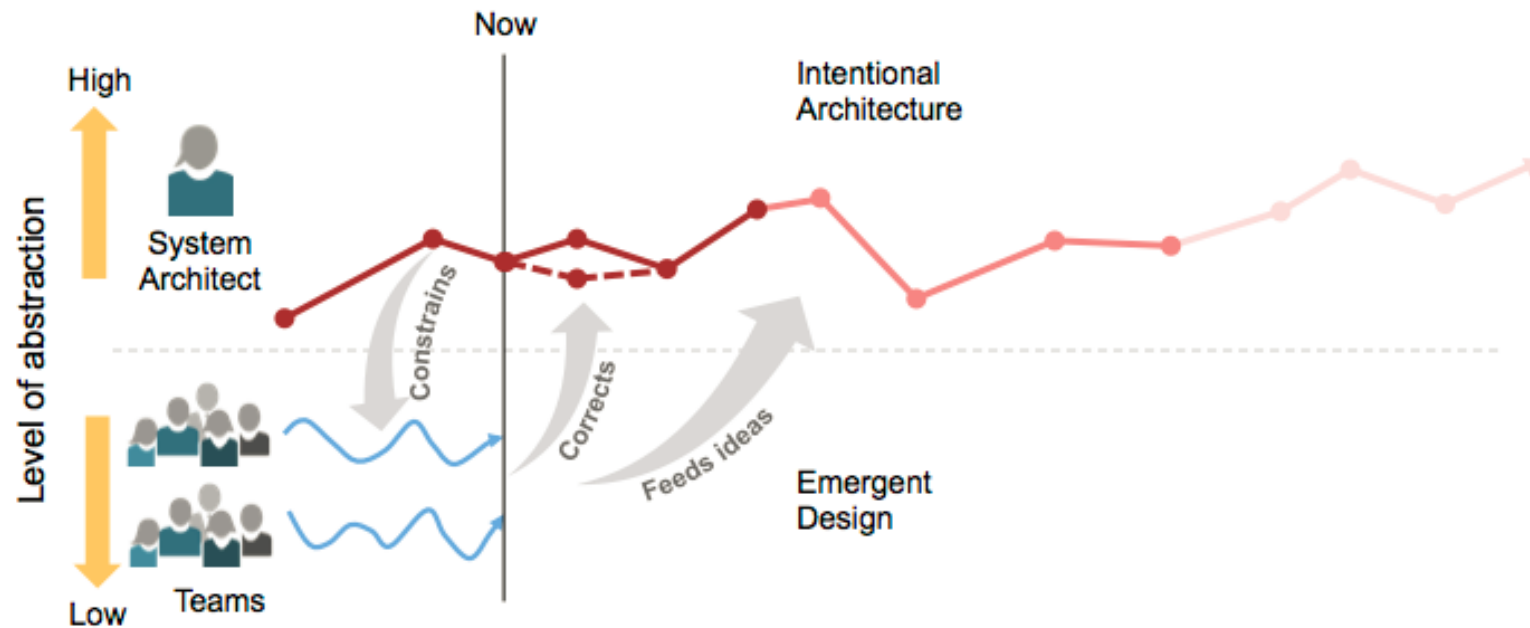
Reality

- Architecture effort on a project is function of complexity and not any particular method. Finding right balance is difficult.
- Create "fit for purpose" architecture - avoid gold plating.



- Architecture isn't finished when the teams start working with it
 - That is OK ...
- It might never get finished ... that is OK too.
- Architecture documentation typically labelled as a “living” document, to be updated/maintained through to production release and beyond



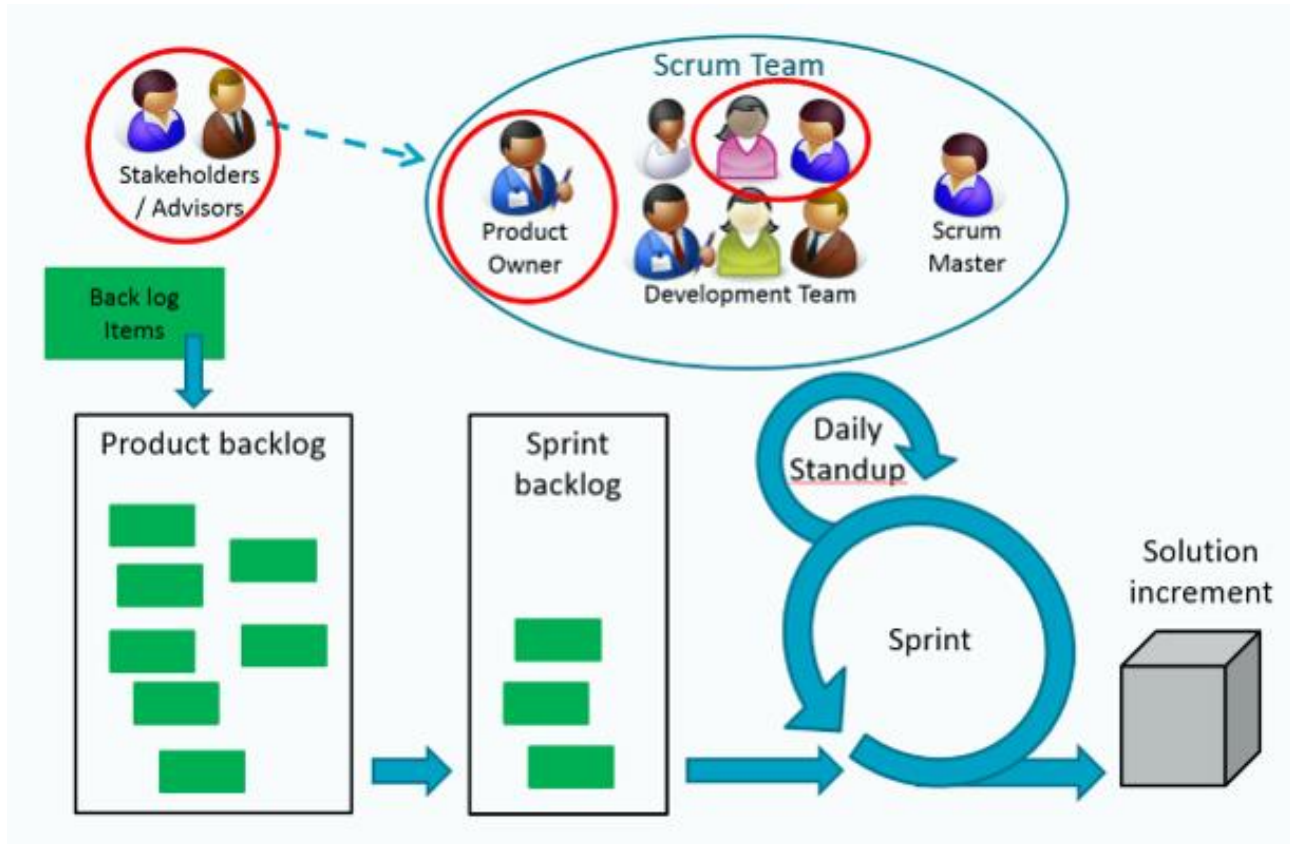


Collaboration model between architects and agile teams (architect as a servant leader)

- “Intentional architecture” constraints “emergent design” by agile teams
- Emergent design informs and adjusts intentional architecture
- Emergent design feeds new ideas to intentional architecture

Architects in agile teams should

- Value **Initial** Architecture Up Front over **Big** Architecture Up Front
- Be collaborative communicator over standards enforcer



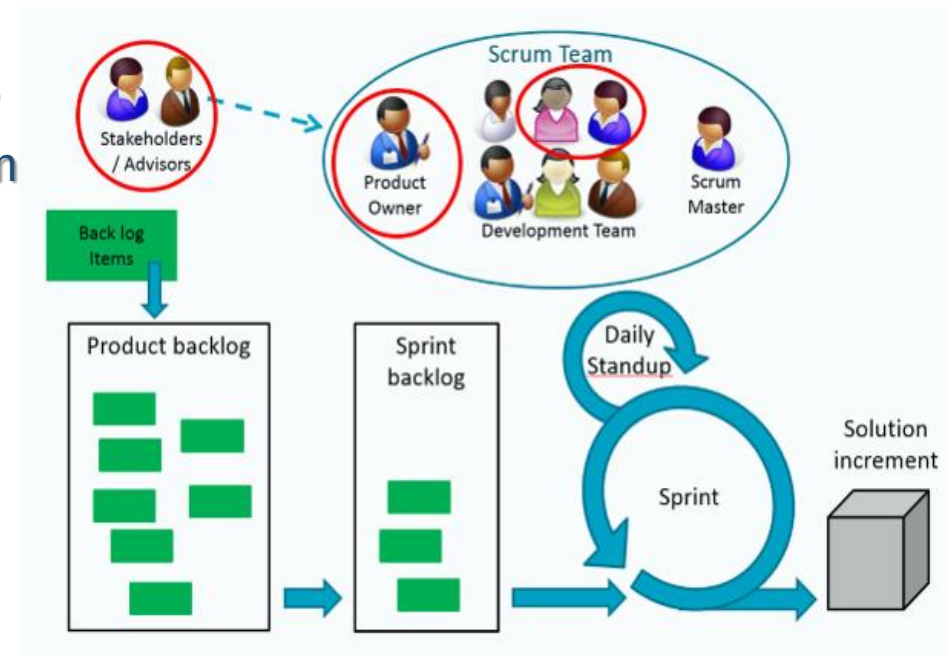
Where does an architect fit in an agile team:

- **Option 1** - Architect is part of the Scrum team taking on tasks that are addressing architecturally significant concerns
- **Option 2** – Architect as Scrum stakeholder looking to influence the Product Owner's decisions about what goes on the product backlog and what is prioritised, to help achieve their architectural objectives.
- **Option 3** – Architect as Scrum Product Owner making the decisions around priorities and the 'definitions of done' for the product or component.



Where does an architect fit in an agile team:

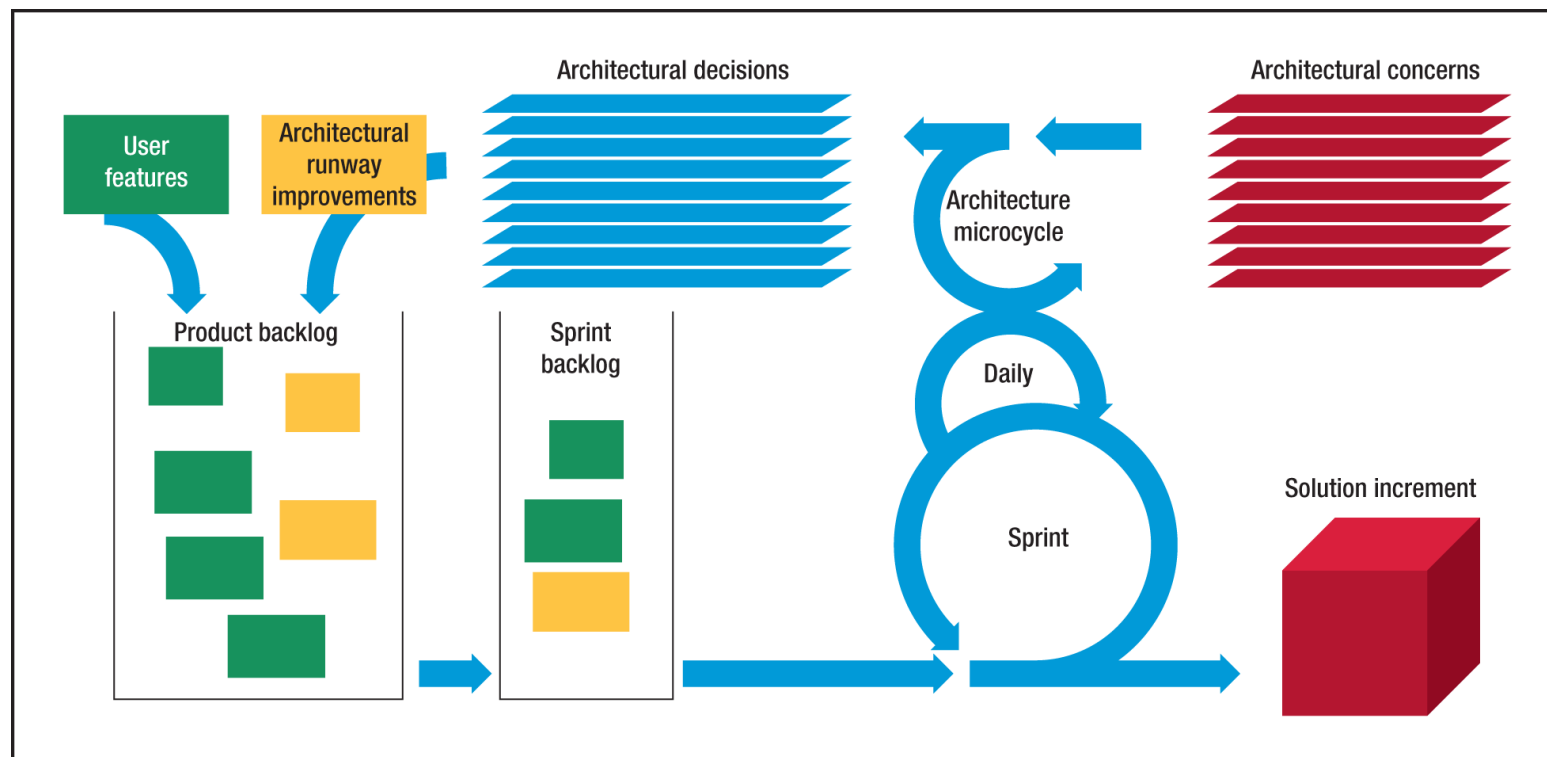
- Early stages of delivery, **continue elaborating the architecture** through discussions with the business, client architecture team and implementation team – driving approvals from the Design Authority. This requires the architect to provide business and technical context and details of historical decision to the delivery team.
- Providing **guidance to the technical delivery team** on expected realisation of architecture to satisfy requirements.
- **Influencing and guiding the Product Owner** on decisions around product backlog, prioritisation and scope to achieve business goals with reasonable and cost effective effort.
- Protect the scope by **describing the implications** of change in terms of architectural impacts and consequential effort and cost terms.
- Attend **daily stand ups** with developers and a **separate** daily stand up **with architects** (for larger engagements).





Managing architecture runway

- Architecture enablers are technical initiatives needed to support development of business functionality
- Architecture enablers emerge either from enterprise architects at the program level or agile teams seeking to improve the existing solution
- Architecture is realised by gradually implementing architecture enablers through agile sprints





Part 1

- Introduction and objectives
- Agile - foundational concepts
- Flavours of Agile
- Architecting Solutions in Agile Environment
- **Envisaging Initial Architecture**

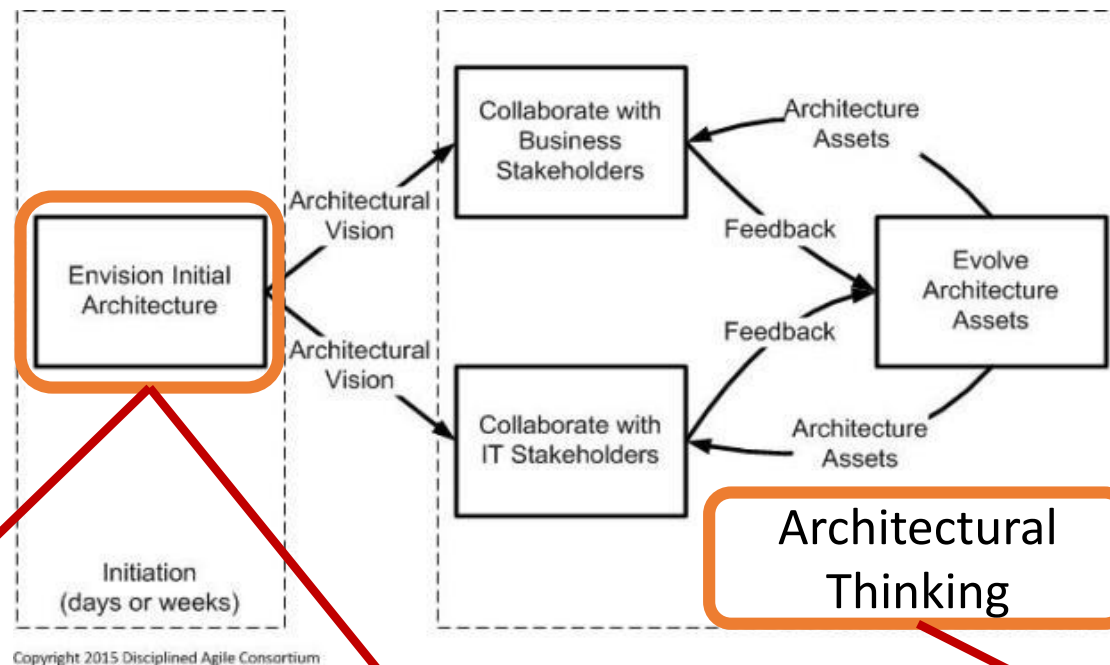
Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- Summary and References

So what do we mean by “...Initial Architecture...”?



- Identify high-level scope
- Identify initial requirements
- Identify and architecture vision



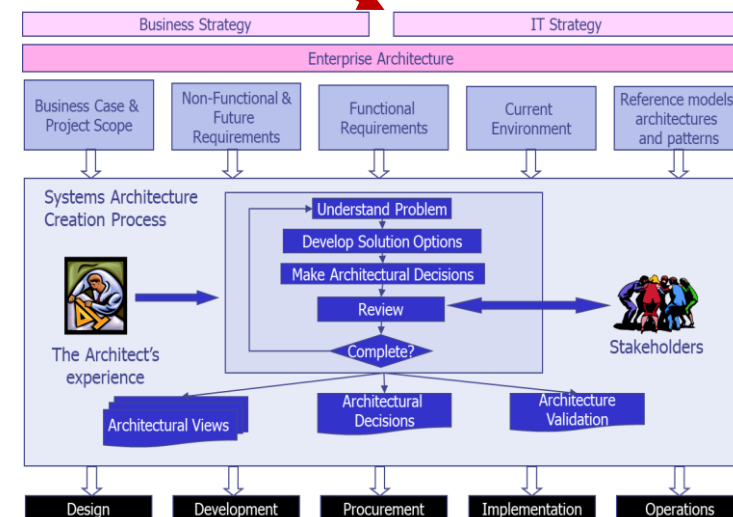
**Design Thinking /
Scoping Workshops
2-3 days**

Initial Requirements

Architecture-on-a-page
with annotations

**Project Initiation /
Feasibility / Sprint 0
1 – 2 weeks**

Initial Architecture

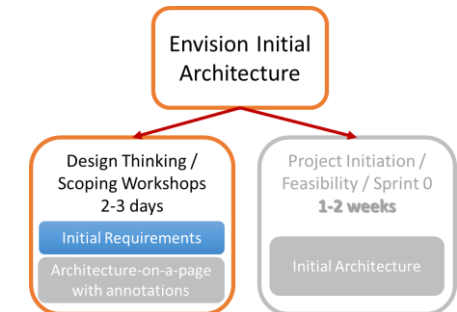


Capture key non-functional requirements



Capture a minimal set of NFRs that will influence key parts of the solution. Capture as bullet points in one slide. If client doesn't know, make assumptions.

Pulse NFR - Capacity							
Summary Usage is expected to be predictable, with predictable peaks and troughs on each day. Typical usage is expected to centre around :							
Category	Sub Category	Reference	Description	Notes	SME	SME Response	Overall Status
Scheduling	Concurrent users	NFR_CAP_0001	Pulse is capable of supporting 350 concurrent planners to manually schedule work orders				TBC
Scheduling	Automation	NFR_CAP_0002	Pulse will have the capacity to automatically plan work 12 months in advance. Initial release only up to 1 month with the final target to be 12 months.				TBC
Authentication	Concurrent users	NFR_CAP_0003	The pulse system should have the capacity to have 2800 concurrent total active users.	Concurrent Users = 40% of total registered users (7000)			TBC
Authentication	Subcontractor throughput	NFR_CAP_0004	The Subcontractor portal should have the capacity to handle 500 concurrent subcontractor portal active users.	Subcontractors make up 10% of the users. Assuming concurrency of 40% of the use for admin users on the subcontractor portal as most subtle 2-3 hours be done towards the end of the day (end of week) Total users: 50% of Total users: 3,500 Current sub-contractors = 350 Total contractors = 350			TBC
Authentication	Mobile throughput	NFR_CAP_0005	The mobile application will be able to have 2,240 active field techs authenticated at any given time in final phase.	Field Techs make up 80% of total registered users. Assuming 40% of this will be currently active on the mobile application. Total users = 80% of 7000 = 5,600 Currently Active = 40% of 5,600 = 2,240			TBC
Work Order	Completed Work Order Task Submission On-Peak	NFR_CAP_0006	The pulse system should have the capacity to receive 12,000 Work Order Task submissions from the mobile application during peak hour	Total work order in a month = 500K For a day: 16,000 Peak hour 25% = 4,000 work orders Assuming 1 work order = 3 work order tasks on average.			TBC
				Field Techs make up 80% of total registered users which is around 5600 field techs. Most of them should be pulling the work around 3 or 4 times a day (minimum once a day). Assuming that 25% of the total requests for the day is made during the peak hour. Total Field Techs = 5,600			
Version Control Key Assumptions System Categorisation Key Targets Performance Capacity Reliability & Recovery Support Security							



What will impact your architectural decisions the most? These are the key areas to capture:

Capacity – number of registered vs concurrent users, peak transactions per second.

Performance – response time for key use cases at the 95th percentile (never 100!).

Reliability & Recovery – high availability, disaster recovery.

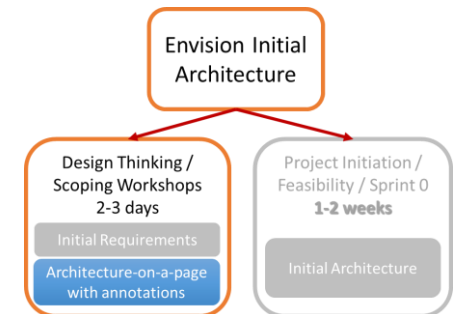
Security – identity, authentication, authorization, encryption at rest

Define architecture-on-a-page



This diagram is used to communicate scope and key elements of the solution to business stakeholders and client architecture team. This is partly a scoping artefact. Simple is better. Used to talk through solution to audiences and triggers discussions around the solution and delivery.

**Unknown is OK
and common**



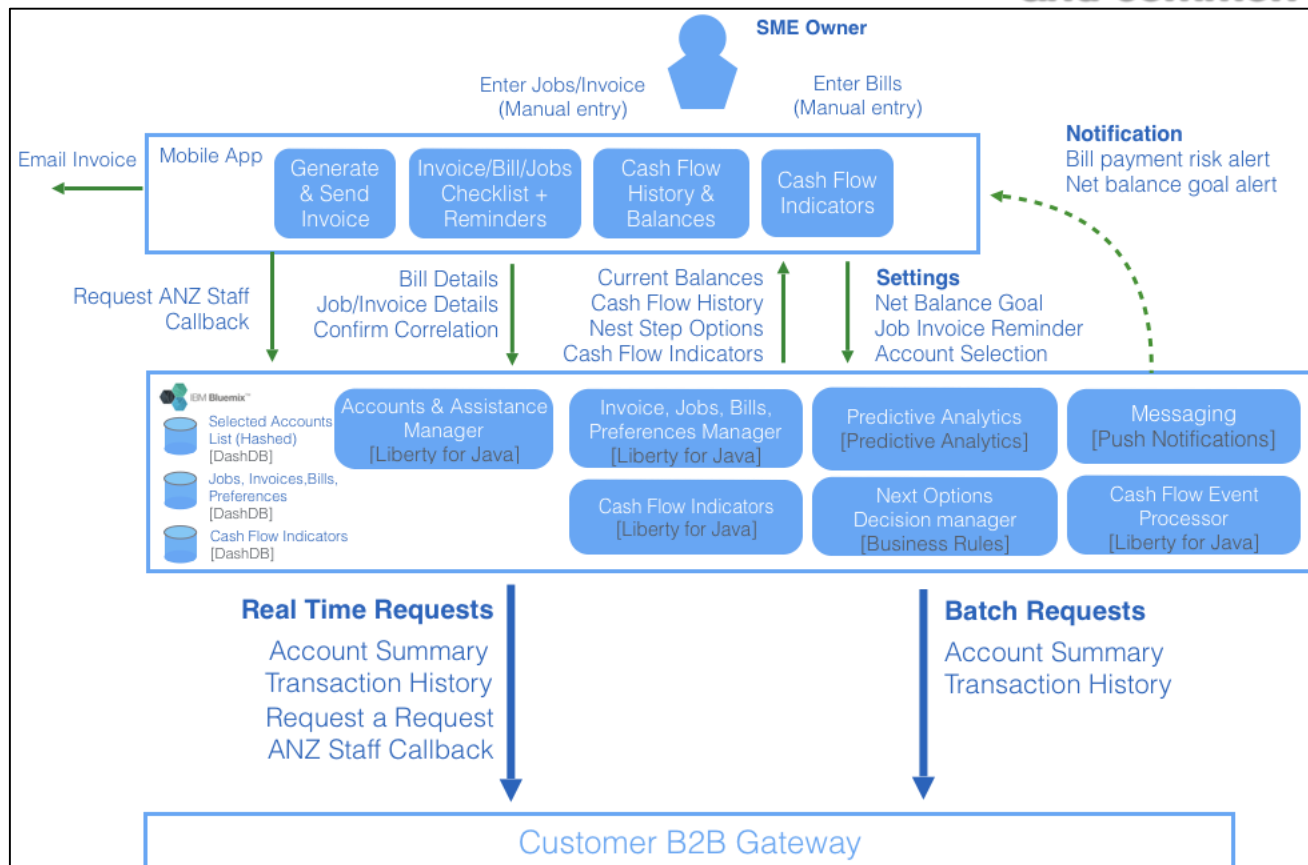
Essential items to capture:

Users – internal/external, the main users.

Components – key software elements that will be procured, developed, impacted.

Integration – call out key integration points for golden path. Not everything, just key ones.

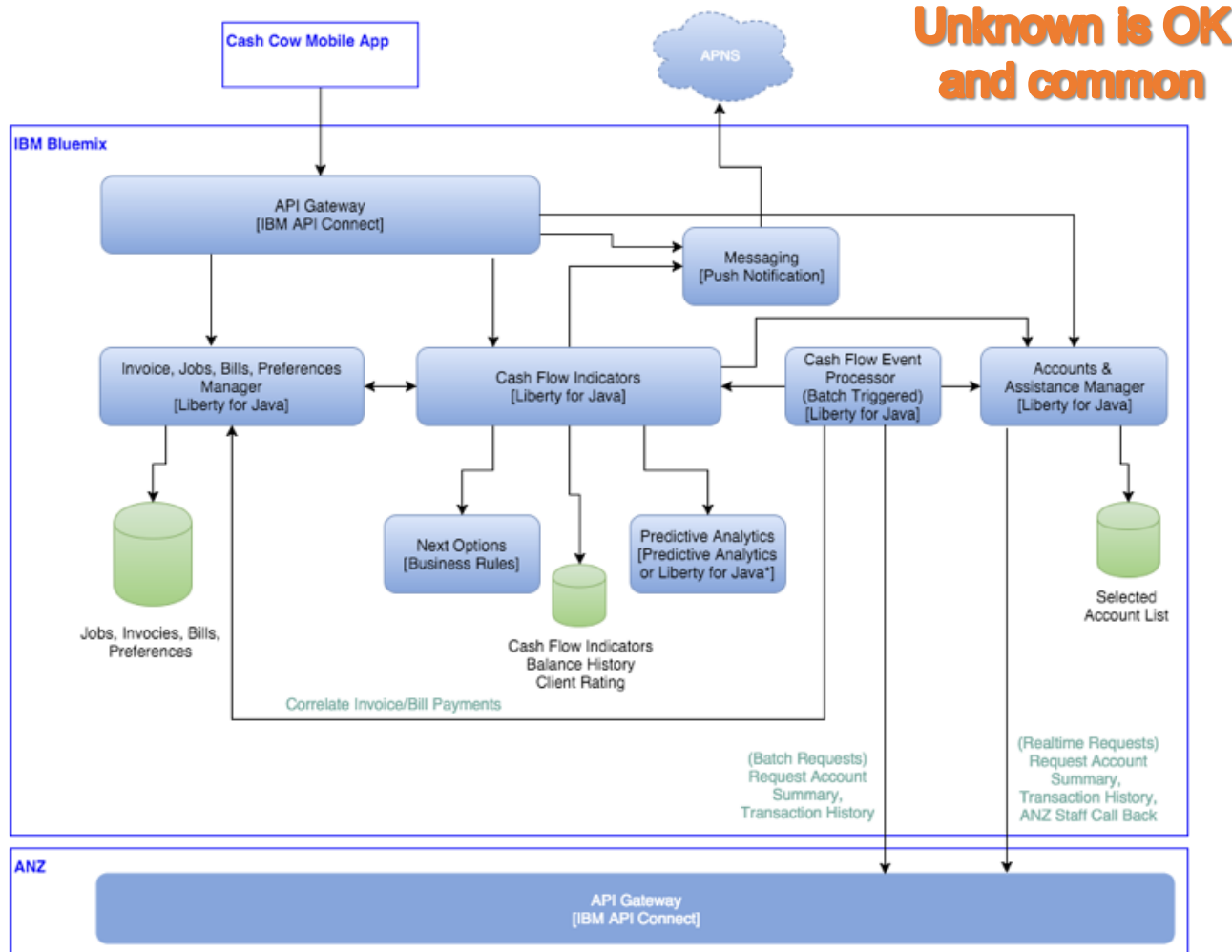
Detail – include logical and/or physical to illustrate scope boundary



Initial Architecture (1/5) – Initial functional viewpoint

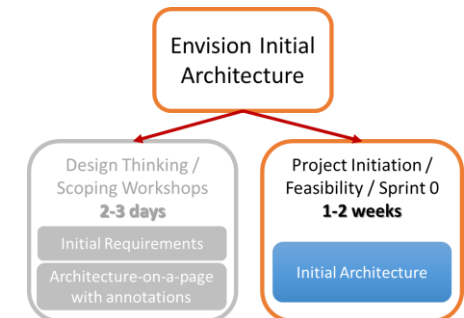


More of the solution is teased out to get agreement from business and client architecture teams.



Essential items to capture:

Component Model – key components and their relationships to provide a more detailed walk through of solution workings.



Initial Architecture (2/5) – Initial functional viewpoint



Description of components should be included as these slides will be distributed to client technical team for review

**Unknown is OK
and common**

Envision Initial
Architecture

Design Thinking /
Scoping Workshops
2-3 days

Initial Requirements
Architecture-on-a-page
with annotations

Project Initiation /
Feasibility / Sprint 0
1-2 weeks

Initial Architecture

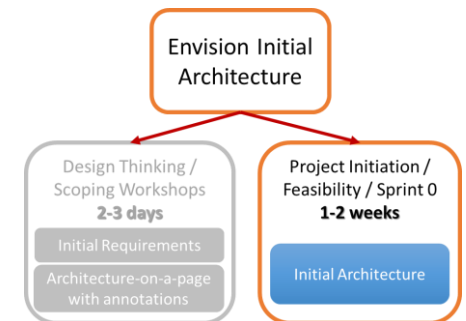
Component	Description	Integration
Invoice, Jobs, Bills, Preferences Manager [Liberty for Java]	Manages the storage and retrieval of Invoices, Jobs, Bills and user preferences (CRUD)	- CRUD API exposed to app via API Connect - Called by Cash Flow Indicators service - Called by Cash Flow Event Processor - Calls Cash Flow Indicators to calculate effects to cash flow indicators
Cash Flow Indicators [Liberty for Java]	- Calculates the cash flow indicators including including cash flow graph data and balance goal progress - Coordinates prediction of cash flow levels for bill payments, supplying historical cash balances, pending bill payments, pending invoices and client rating to the predictive analytics service via REST API call - Initiates push notification for cash flow alert condition and net balance goal alert	- API exposed to app via API Connect to provide cash flow indicator values - Called by Cash Flow Event Processor - Called by Invoice, Jobs, Bills Manager - Calls Messaging service to send notification on alert condition (action determined from business rule) - Reads cash flow data from database and calls Predictive Analytics syndicated service to obtain prediction results - Persists prediction results back to database
Cash Flow Event Processor [Liberty for Java]	- Periodically requests account summary and transaction history for all clients (nightly) - Determines new transactions and calls Cash Flow Indicators to calculate effects to cash flow indicators - Correlates transactions to pending invoices and bills to mark them off when paid - Retrieve initial 2 years and derive regular payments	- Calls ANZ B2B Gateway for account summary and transaction history - Calls Cash Flow Indicators - Calls Invoice, Jobs, Bills, Preferences Manager
Predictive Analytics [Predictive Analytics]	Utilises historical view of incoming and outgoing daily totals to predict future cash flow	Called by Cash Flow Indicators
Messaging [Push Notifications]	- Register push notification registrations - Send push notification (eg alert condition requires attention)	- Called by Cash Flow Indicators - Called by API Connect for device push registration - Sends push notifications directly to APNS
Accounts & Assistance Manager [Liberty for Java]	- Retrieves account summary and supports user business account selection - Calculates aggregated cash and credit balance on selected accounts - Detects account closure and new accounts - Provides list of selected accounts - Account IDs are hashed	- API exposed to app via API Connect to provide aggregated cash and credit balances and request ANZ Staff Call Back - Called by Cash Flow Indicators to return current day transactions (in order to calculate incoming/outgoing totals) - Adds context to ANZ Staff Call Back for Cash Flow position - Called by Cash Flow Event Processor to return selected accounts
Auth Proxy [Liberty for Java]	Inserts a client_id attribute in the HTTP header when calling the Authentication API exposed by ANZ API. All other attributes, payload and response are proxied without change.	- Bluemix API Connect calls Auth Proxy during authentication step of OAuth 2.0 Resource Owner Password Credential flow. - Auth Proxy forwards authentication requests to ANZ API Connect

Initial Architecture (3/5) – Initial operational viewpoint



Infrastructure can include cloud and on premises parts of the solution. Start with environments at the high level and then cover lower level infra as required by discussions with client technical teams.

**Unknown is OK
and common**



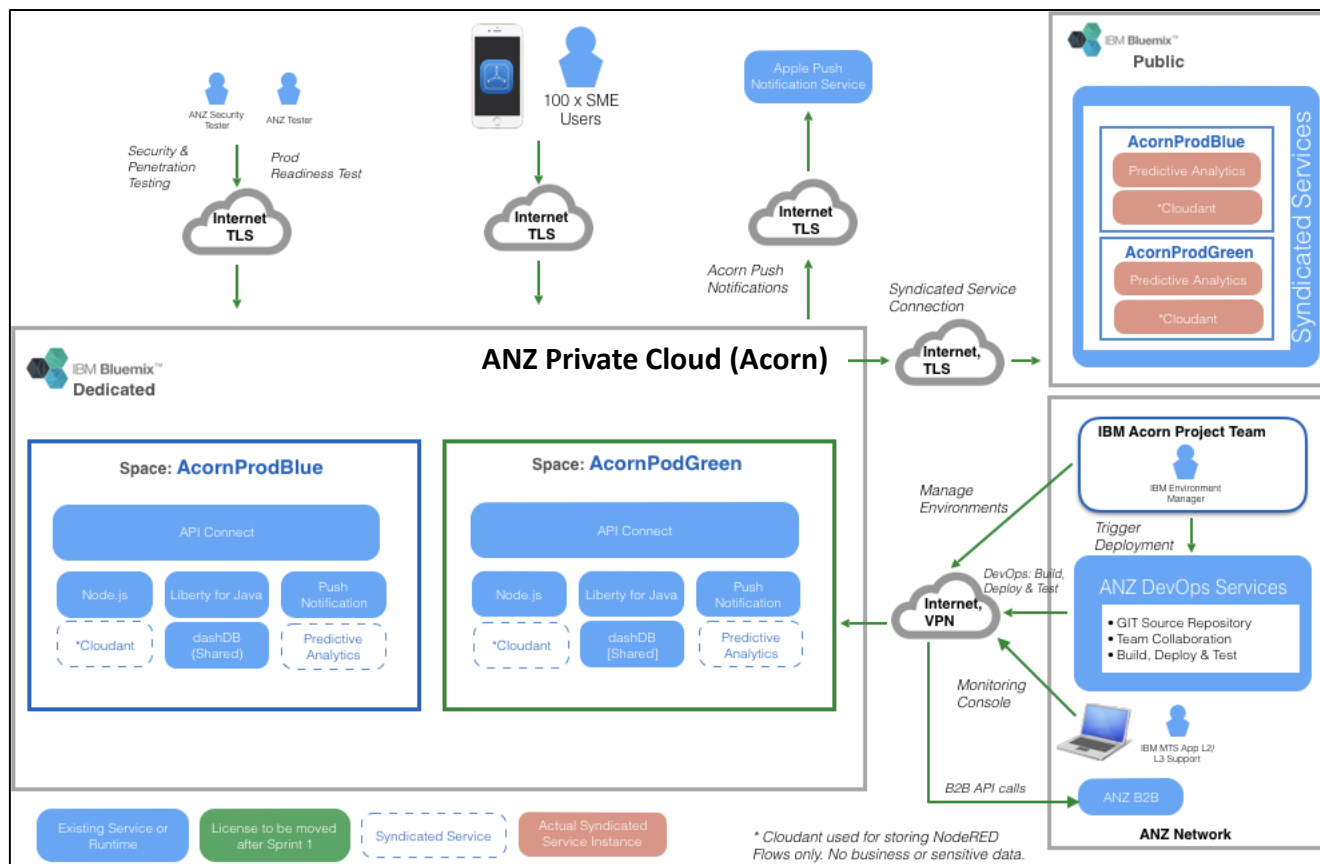
Operational/Infrastructure (Logical):

Essential items to capture:

Environments – capture the development, test and production environments.

Redundancy – If disaster recovery is required include dual sites with links between them for data replication if required.

Connectivity - include integration to DevSecOps, remote or local developers and testing, distribution.

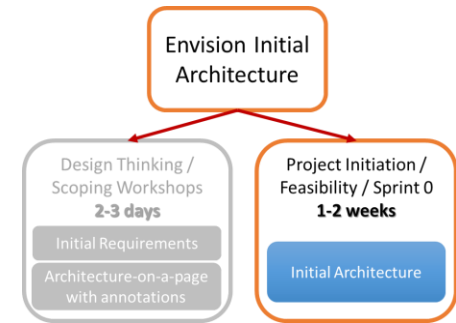
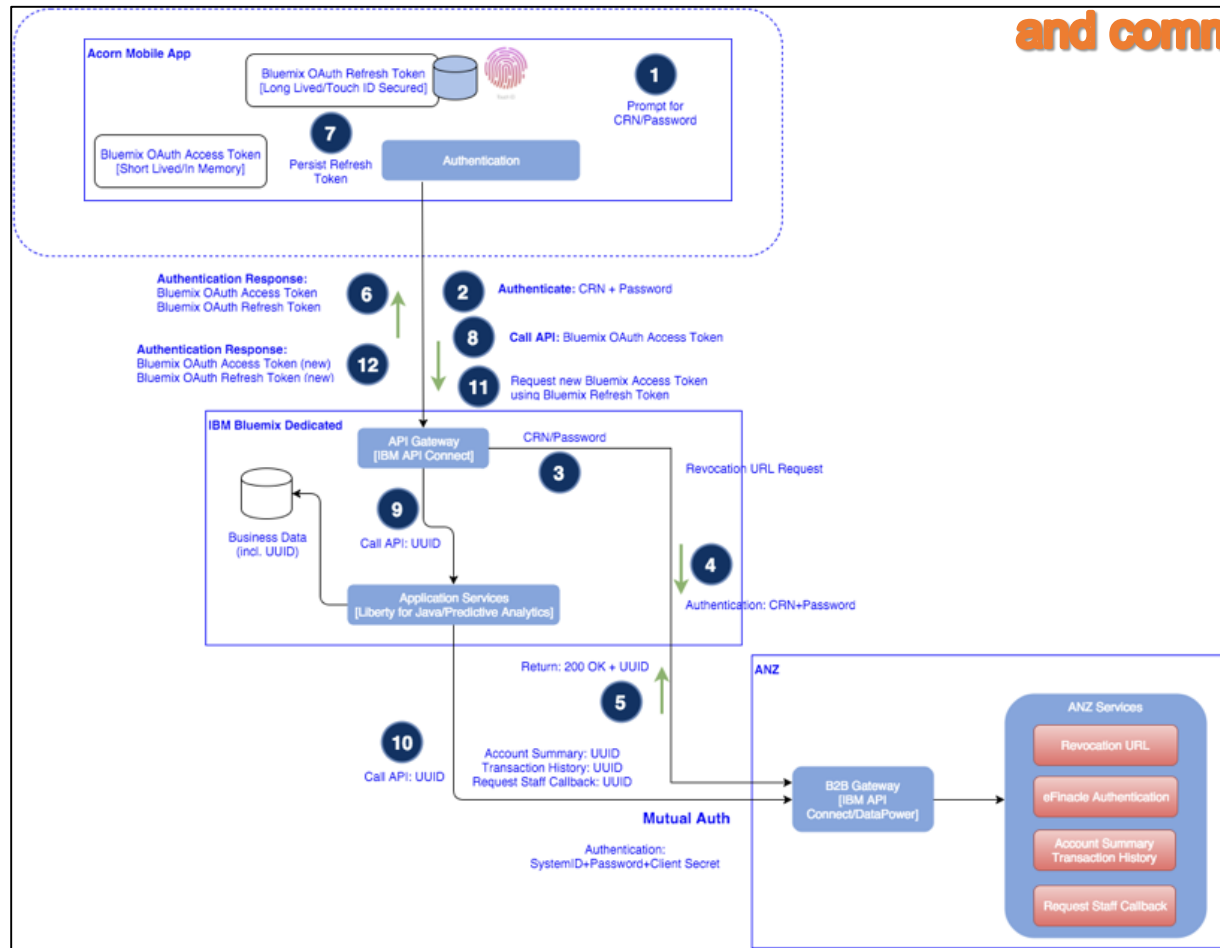


Initial Architecture (4/5) – Initial security viewpoint



If security is a major factor (as it should be!) a walk through of the main authentication flow should be captured to satisfy client security team. It doesn't have to cover 100% of cases, just the key concerns.

Unknown is OK
and common



Essential items to capture:

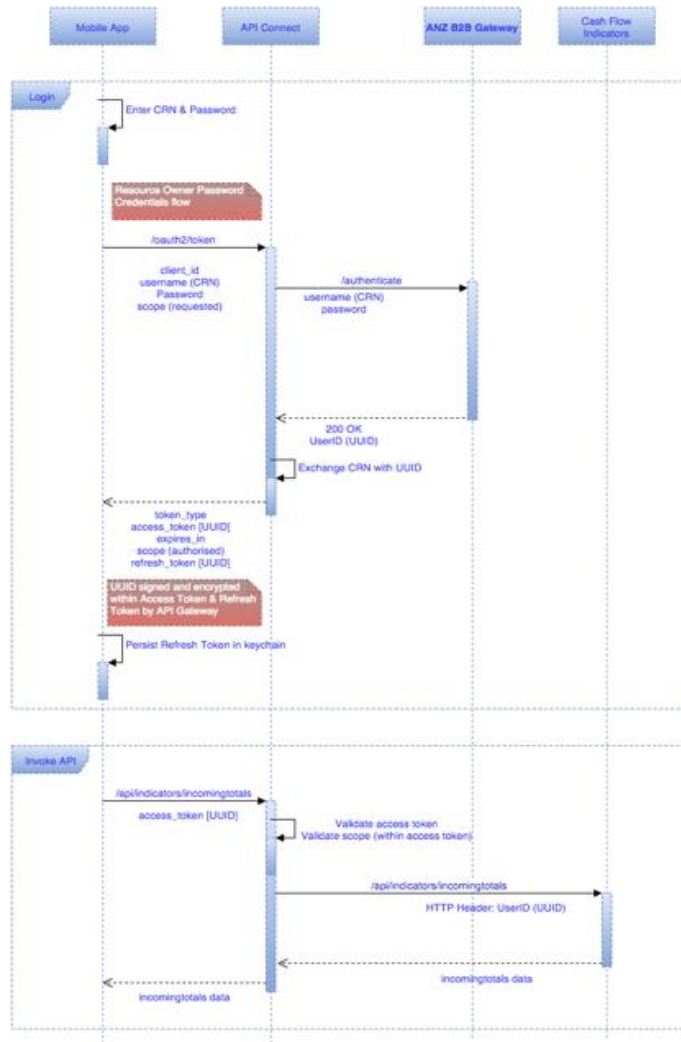
Security Architecture – include key steps for authentication and identity.

You will, guaranteed, change this sequence many times during delivery. The point here is to satisfy client security team that their primary security concerns can be addressed with the proposed solution. This is typically a top priority at financial institutions.

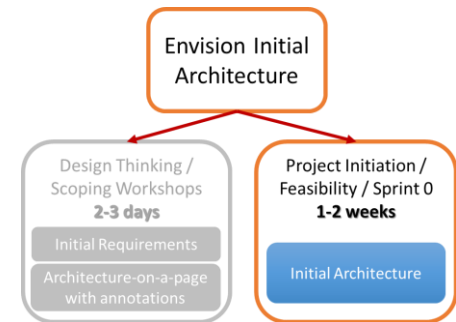
Initial Architecture (5/5) – Initial security viewpoint



A sequence diagram may also be required for complex flows. This should be at a level that addresses primary security concerns. No need to cover everything.



Unknown is OK
and common



Essential items to capture:

Security Architecture – include key steps for authentication and identity.

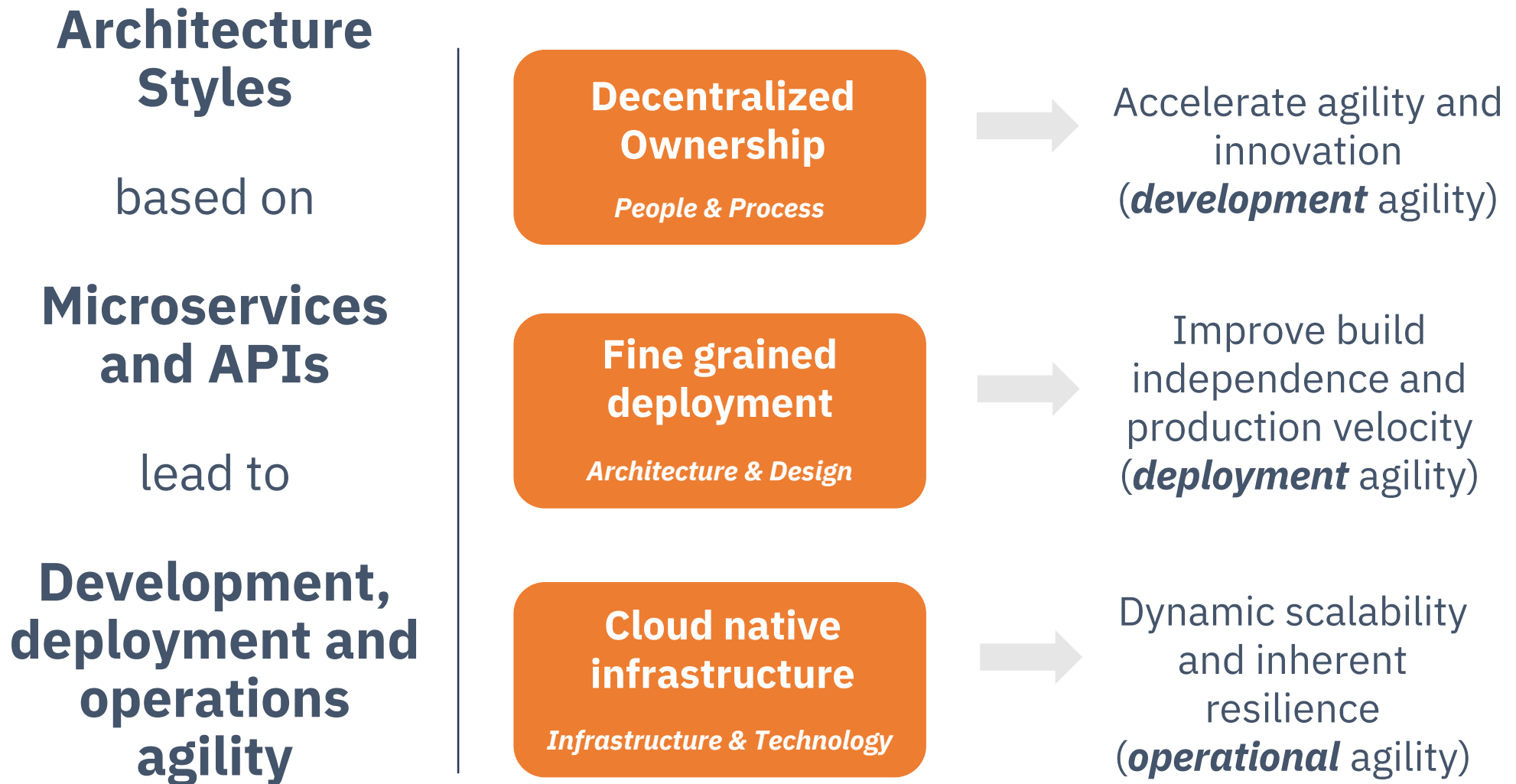


Part 1

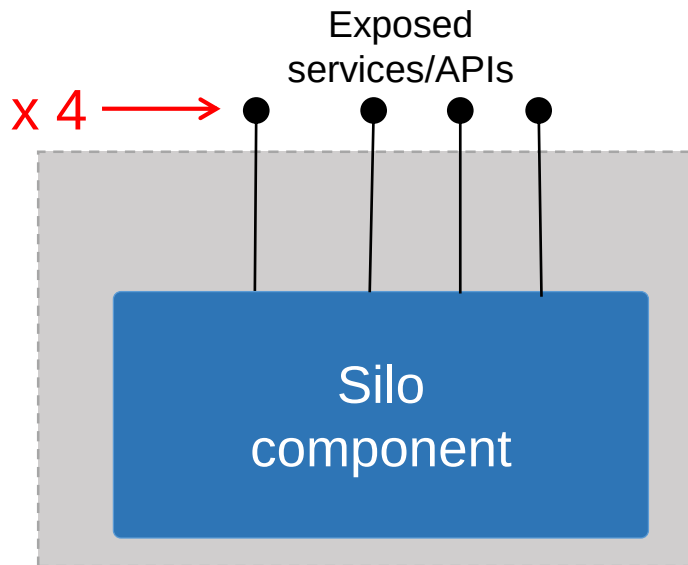
- Introduction and objectives
- Agile - foundational concepts
- Flavours of Agile
- Architecting Solutions in Agile Environment
- Envisaging Initial Architecture

Part 2

- **Impact of Agile on Architectural Styles**
- Agile and DevOps
- Summary and References



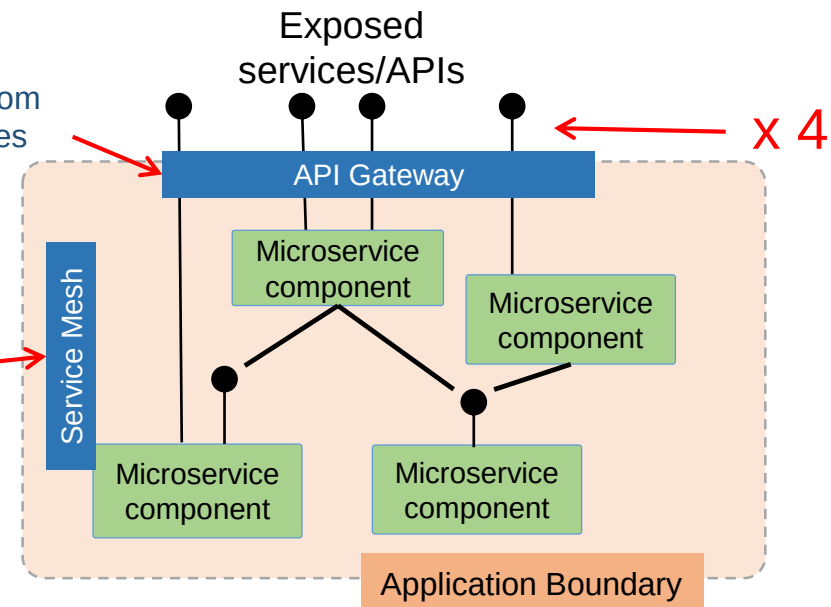
Traditional vs Contemporary Architectural Style



Monolithic application

API Management focuses to how APIs from a group of microservices are exposed outside application boundary.

Service Mesh provides services routing within the application boundary.



Microservices application

“micro” refers to the granularity of the components, not the granularity of the exposed interfaces

Microservices are not just more fine grained web services and APIs are not Microservices

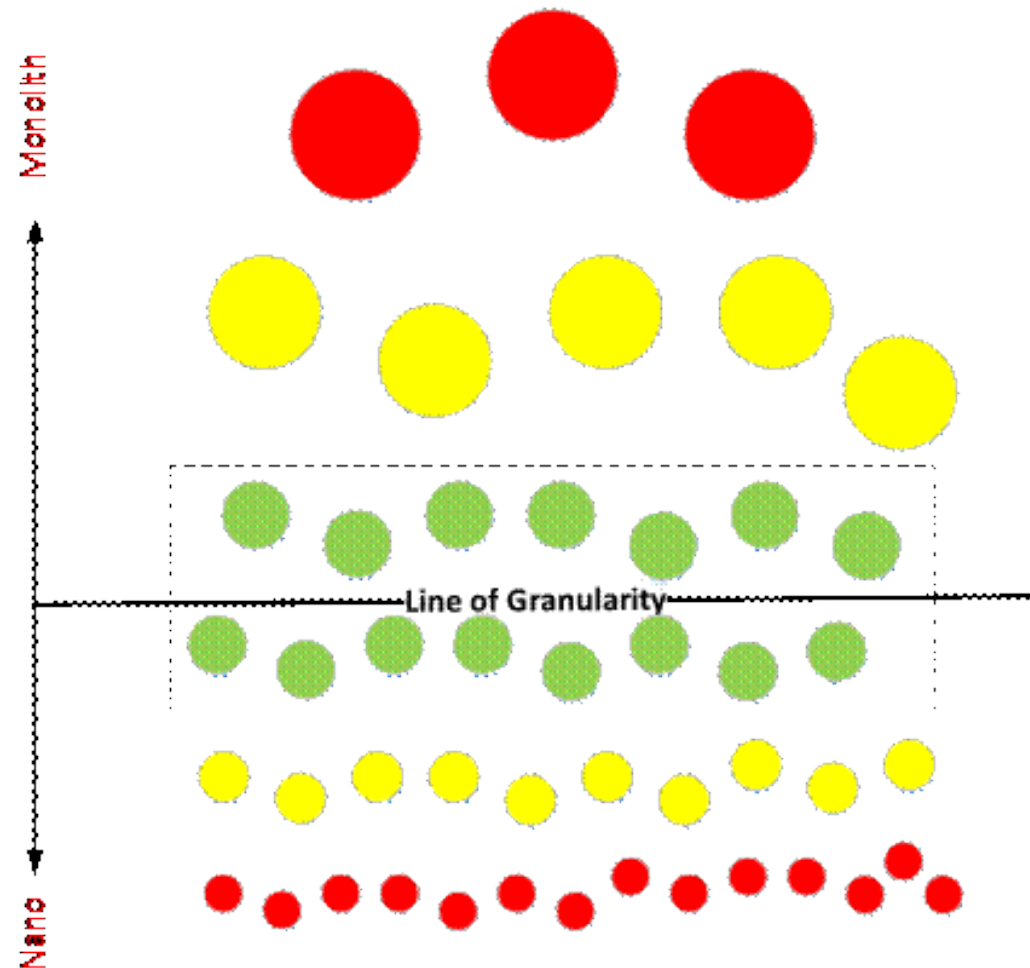


Problems with monoliths include:

- Even small, minor changes require rebuilding of the entire code base and re-deployment of the new build.
- Change cycles (for various functions and features) will have to be tied together, causing an undesirable dependency.
- Achieving modular structure within a monolith is hard to enforce.
- Scaling is achieved by replicating the entire application (though specific functions may have different scalability requirements).

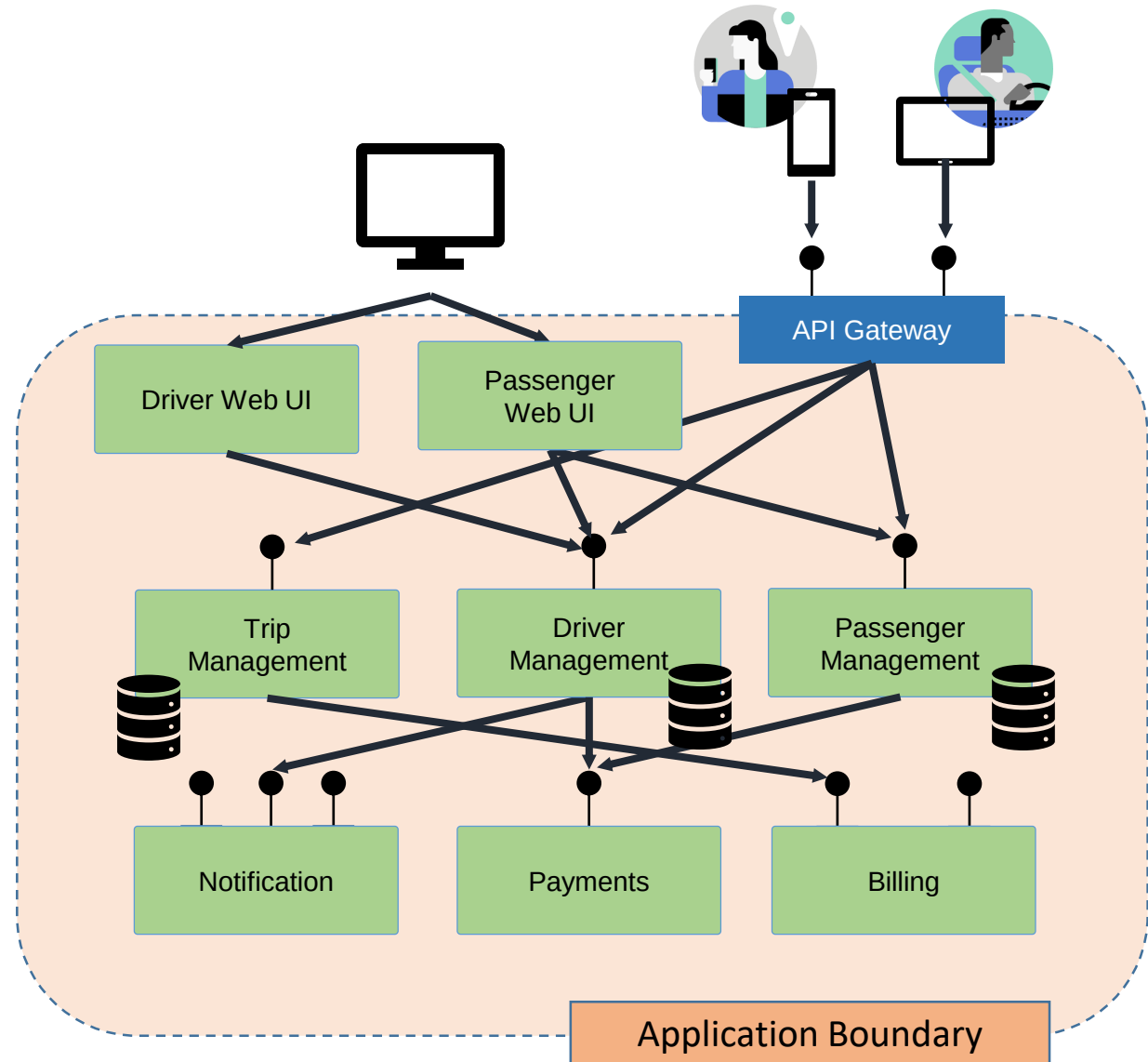
Problems with nano-services include:

- Remote calls are expensive (from a performance perspective).
- Communication between services becomes chatty, resulting in a sub-optimal system.
- Unmanageable explosion of services can result in service proliferation, challenging governance.





Q: Is microservice granularity correct here: too coarse grained, too fine grained or just right?





Decentralized Ownership

People & Process



Accelerate agility and innovation
(*development* agility)

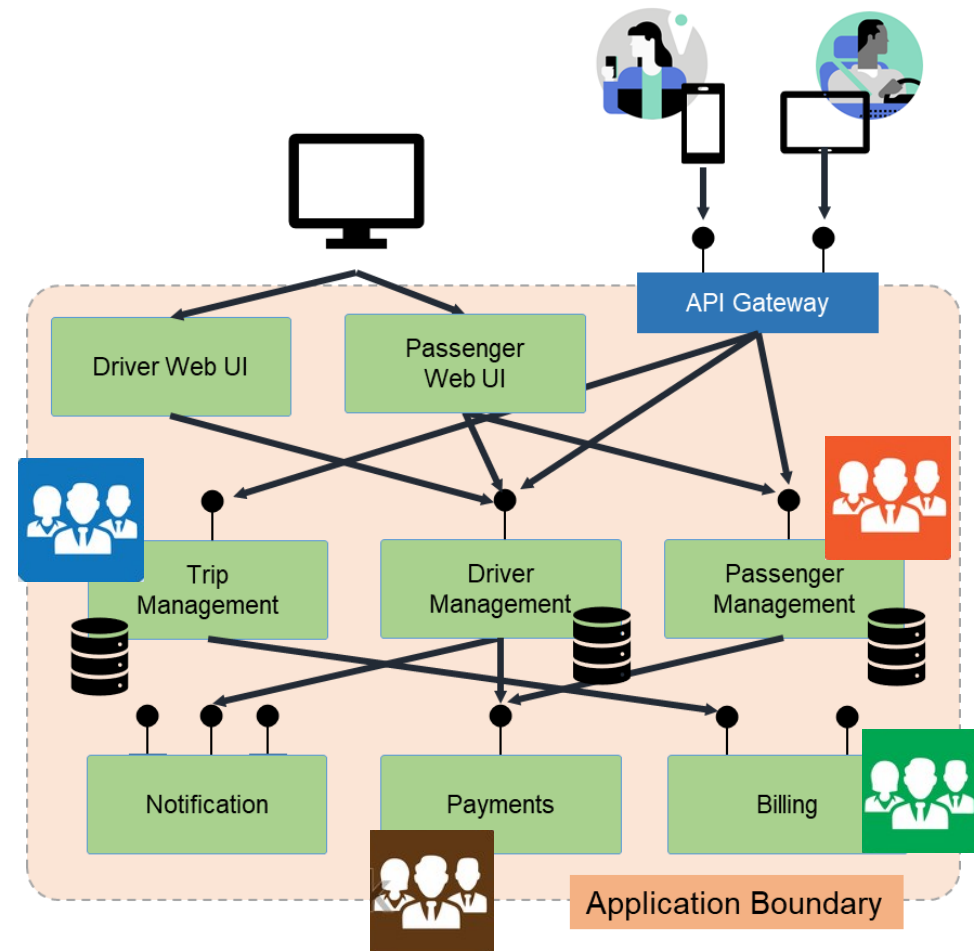
Inverse Conway Manoeuvre

Align teams to microservices components, and create clear delineation of responsibilities.

- The teams need to have cross-cutting skills since their goal is to deliver business results, not technology.
- Result is less handoffs between teams to accomplish a business objective.

=====

- The practicing architect is part of a team and responsible for execution of the individual team mission as well as the related governance requirements that cut across multiple teams.





Fine grained deployment

Architecture & Design



Improve build independence and production velocity (**deployment** agility)

Agility:

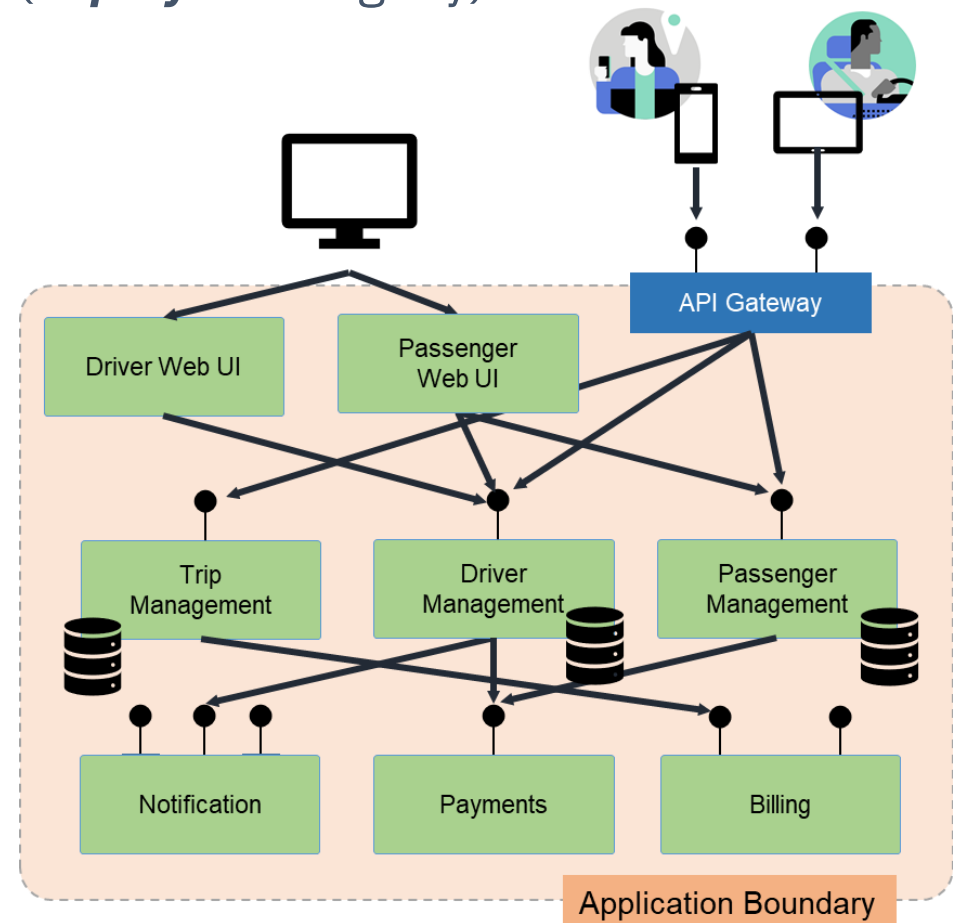
- Individual microservices can be changed, rebuilt, and deployed independently of others enabling safer application of changes and maximizing speed to production.

Scalability:

- Individual microservices can be scaled on their own, allowing you to take advantage of efficient elastic scaling of cloud infrastructures.

Resilience:

- Isolated microservices that are deployed in separate containers cannot affect one another by stealing shared resources, such as memory, connections, or CPU.





Cloud native infrastructure

Infrastructure & Technology



Dynamic scalability
and inherent
resilience
(*operational* agility)

Elastic scalability:

- Microservices are scaled horizontally and allocated on-demand in a cloud-like infrastructure.

Disposable and re-creatable:

- Using lightweight container technology encourages changes to be made by redeploying amended images rather than by nurturing a running server.

Starts and stops in seconds:

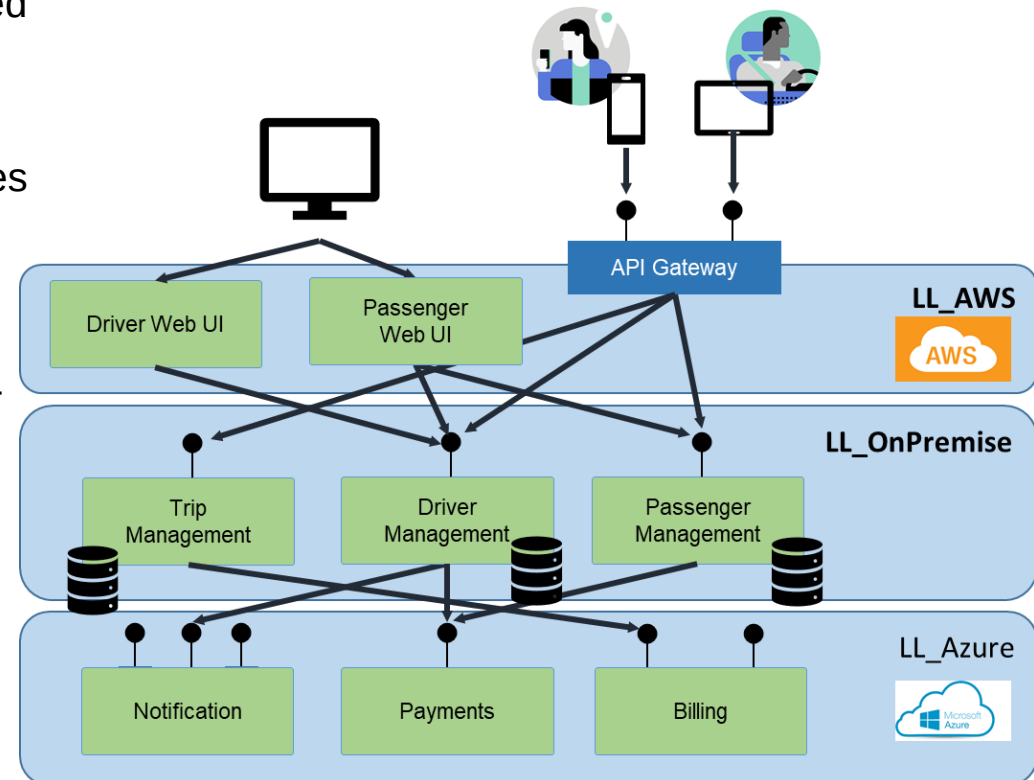
- Microservices are run and deployed as more fine-grained entities and, therefore, take less time to start.

Minimal Interdependencies:

- Unrelated microservices are not grouped. Functional and operational characteristics create colocation and grouping.

Infrastructure as Code:

- Resources and code are declared and deployed together.





Part 1

- Introduction and objectives
- Agile - foundational concepts
- Flavours of Agile
- Architecting Solutions in Agile Environment
- Envisaging Initial Architecture

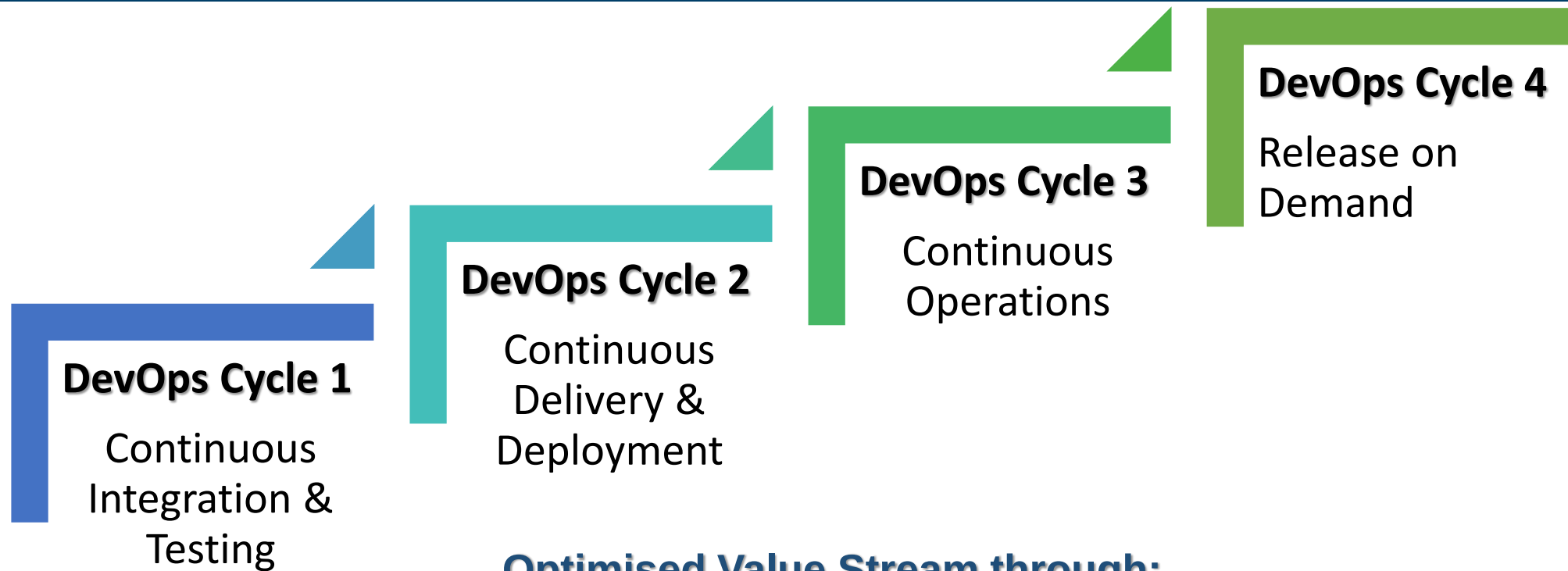
Part 2

- Impact of Agile on Architectural Styles
- **Agile and DevOps**
- Summary and References

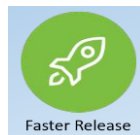


- DevOps is the practice, culture of Development and Operations engineers participating together in the entire service lifecycle, from design through the development process to production support.
- DevOps is also characterized by operations staff making use of the same techniques as developers for their systems work.
- The contraction of “Dev” and “Ops” refers to replacing siloed Development and Operations to create multidisciplinary teams that now work together with shared and efficient practices and tools.

DevOps isn't
a job title, a toolset,
or a marketplace.



Optimised Value Stream through:



Faster Release

Speed of Delivery

Responsiveness to business demand for new features and functionality through **automation** and streamlining delivery.



Lesser Bugs

Improved quality

Reduced opportunity for defects due to **automated** testing.

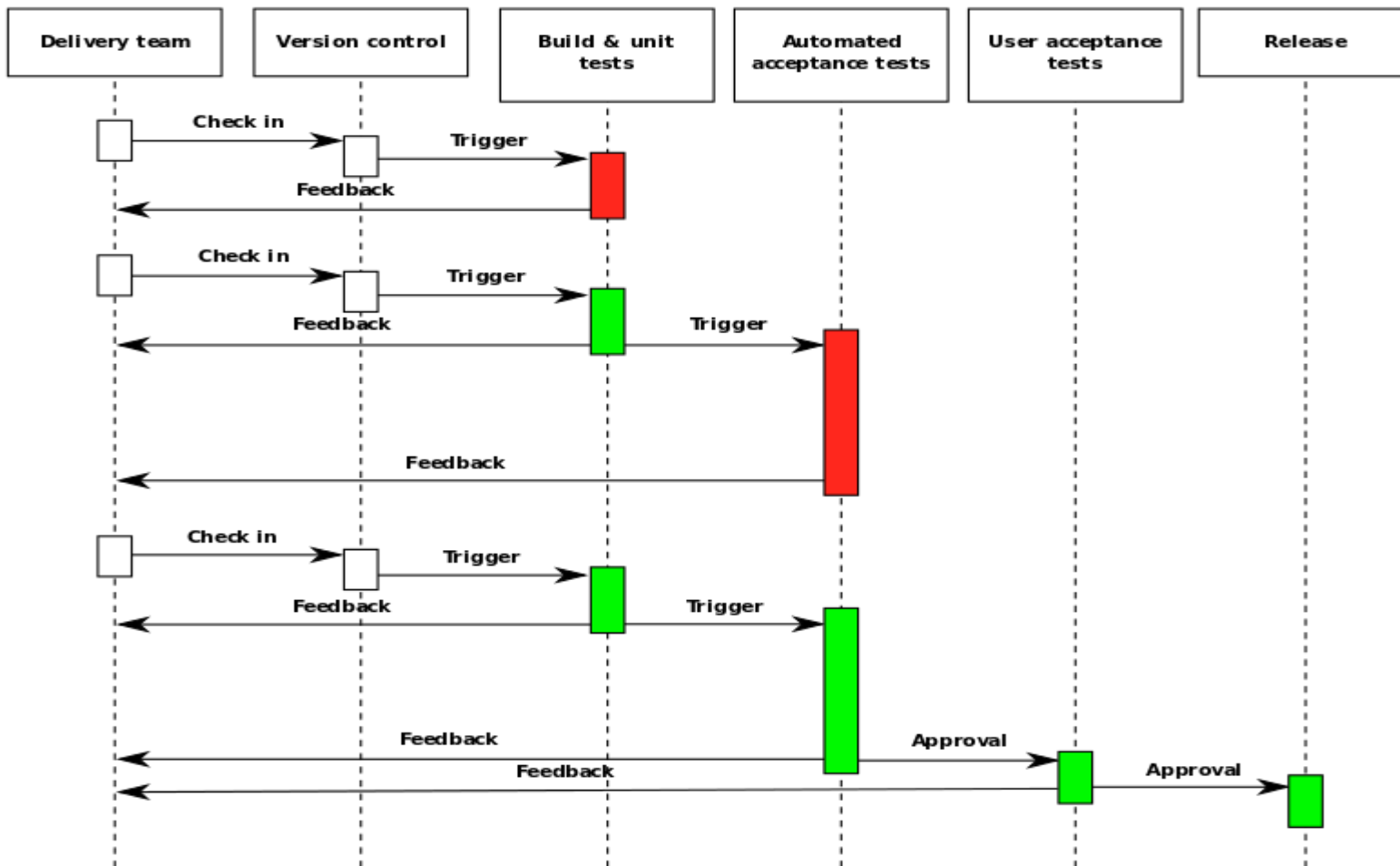


Better Quality

Increased efficiency and capacity

Eliminated waste and rework, enabling the same size team to complete more work driven by integration **automation**.

Deployment pipeline



Continuous delivery is enabled through the deployment pipeline:

- **Visibility** – All aspects of the delivery system including building, deploying, testing, and releasing are visible to every member of the team to promote collaboration.
- **Feedback** – Team members learn of problems as soon as possible when they occur so that they are able to fix them as quickly as possible.
- **Continually deploy** – Through a fully automated process, you can deploy and release any version of the software to any environment.

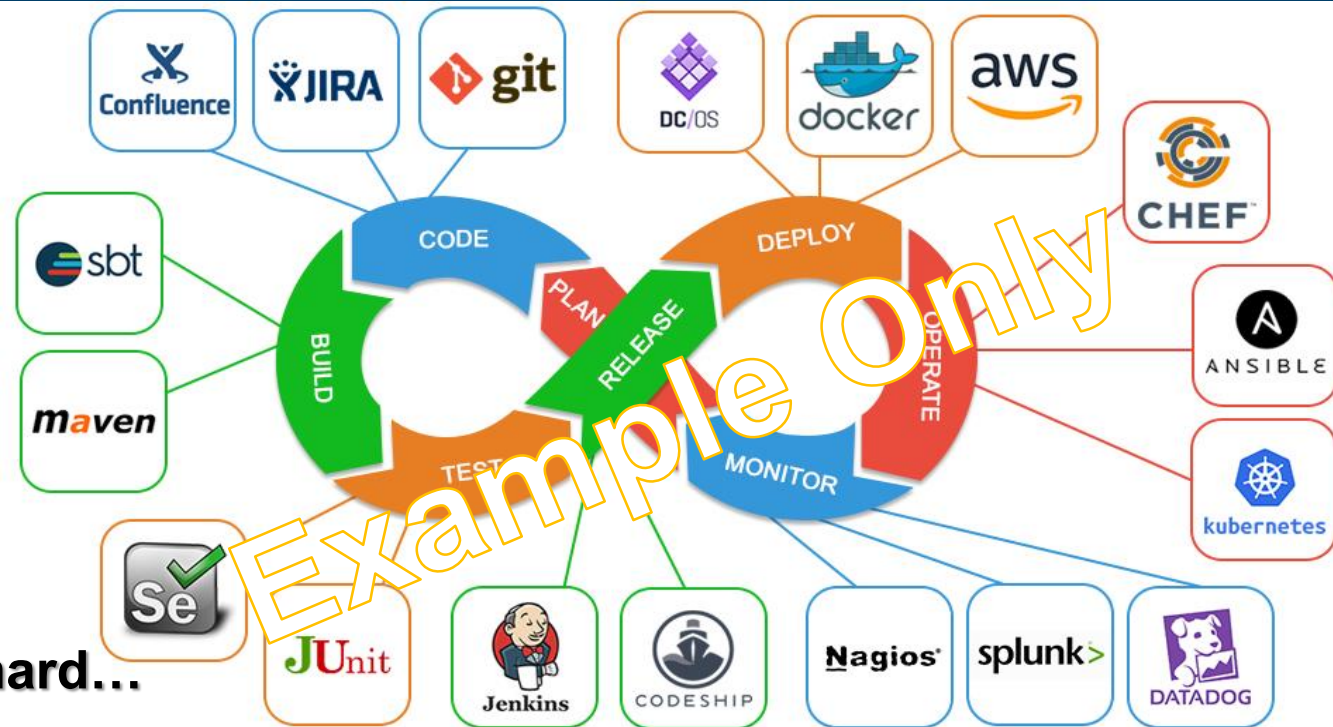


Main categories of DevOps tools

- Version control
- Build and deploy
- Functional and non-functional testing
- Platform provisioning and change management

Selecting DevOps tools is hard...

- Access to tools by broad user base – developers, IT operations, project managers and testers - open up rich collaboration opportunities
- Favour tools with APIs - aim for composition of new capabilities from multiple API-driven tools
- Start small - Prefer achievable gains now over possible future state
- Encourage learning – hold regular show-and-tell sessions from peers and senior staff to demonstrate tools and techniques.
- Avoid special 'singleton' tools – for example avoid “production only” tool due to high licencing costs





Part 1

- Introduction and objectives
- Agile - foundational concepts
- Flavours of Agile
- Architecting Solutions in Agile Environment
- Envisaging Initial Architecture

Part 2

- Impact of Agile on Architectural Styles
- Agile and DevOps
- **Summary and References**



- Agile often fixes time and cost while scope is “negotiable”
- Architecture effort on a project is function of complexity and not any particular method. Finding right balance is difficult.
- In agile environment:
 - Envision initial architecture quickly and evolve gradually
 - Collaboration between architects and agile teams is essential two way street
 - Use Design Thinking in the early stages of the lifecycle to capture essence of user needs and potential solutions
- Architecture enablers are technical initiatives needed to support development of business functionality
- Architecture is realised by gradually implementing architecture enablers through agile sprints
- Value Initial Architecture Up Front over Big Architecture Up Front
- Architecture Styles based on Microservices and APIs lead to development, deployment and operations agility
- DevOps is the practice, culture of Development and Operations engineers participating together in the entire service lifecycle



- Advancing the Practice of Agile - <https://www.agilealliance.org/>
- Agile Software Development - https://en.wikipedia.org/wiki/Agile_software_development
- SAFe website - <https://www.scaledagileframework.com>
- API Management versus Service Mesh - <https://developer.ibm.com/apiconnect/2018/11/13/service-mesh-vs-api-management/>
- Agile Integration Architecture - <https://www.ibm.com/cloud/agile-integration-architecture>
- DevOps Tutorial - <https://www.guru99.com/devops-tutorial.html>
- Flavors of Agile - <https://www.thekua.com/atwork/2018/07/flavours-of-agile/>
- DevOps vs Agile: What's the Difference and How Are They Related? - <https://www.bmc.com/blogs/devops-vs-agile-whats-the-difference-and-how-are-they-related/>
- *Microservices*, Wikipedia, <https://en.wikipedia.org/wiki/Microservices>

THANK YOU



IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both. If these and other IBM trademarked terms are marked on their first occurrence in this information with a trademark symbol (® or ™), these symbols indicate U.S. registered or common law trademarks owned by IBM at the time this information was published. Such trademarks may also be registered or common law trademarks in other countries. A current list of IBM trademarks is available on the Web at "[Copyright and trademark information](#)." Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and its affiliates.

Linux is a trademark of Linus Torvalds in the United States, or other countries, or both. Microsoft, Excel, PowerPoint, Project, Visio, Word, and Windows are trademarks of the Microsoft Corporation in the United States, other countries, or both. UML is a registered trademark of Object Management Group, Inc. in the United States and/or other countries. TOGAF and UNIX are registered trademarks of The Open Group in the United States and other countries. Other company, product, and service names may be trademarks of IBM or other companies. Citrix is a trademark of Citrix Systems, Inc. and one or more of its subsidiaries, and may be registered in the United States Patent and Trademark Office and in other countries.